



中国研究生创新实践系列大赛
“华为杯”第二十届中国研究生
数学建模竞赛

学 校 华东交通大学

参赛队号 23104040152

1.刘 鹏

队员姓名 2.熊宇铭

3.吴志勇

中国研究生创新实践系列大赛

“华为杯”第二十届中国研究生

数学建模竞赛

题目：基于多特征融合 ConvLSTM 模型的强对流天气预测研究

摘 要：

强对流天气局地性较强，空间尺度较小，持续时间或者生命期比较短，却能在短时间内释放出强大的力量，破坏性比较强，是具有重大杀伤性的灾害性天气之一。强对流天气发展比较快，消散得也快，在所有的天气类型中最难预报，同时，强对流天气与一般天气相比，可供研究的个例更少，预报难度更大。基于双偏振雷达数据建立强对流降雨的预测模型，提高预报的准确性，对预防和缓解强对流天气灾害有着重要的意义。

问题一要求实现雷达回波响应 Z_H 的预测，本质上是**时空序列预测问题**。模型需要在学習数据空间分布特征的同时，理解数据的时序发展规律。本文以 **ConvLSTM 网络** 为主干结合**特征层数据融合策略**，设计了 **DTConvLSTM 深度学习模型**。通过迭代寻练权重以及超参数优化，得到了能够拟合 Z_H 发展趋势的深度学习模型。

问题二要求改善雷达回波响应 Z_H 结果，本质上是**时空序列预测模型的优化问题**。本文对训练过程和网络结构同时入手，改进了**损失函数**并增加了**通道空间注意力(CBAM)**，使得模型对高强度回波响应更加敏感；提升了模型自动捕捉关键信息的能力。

问题三要求利用题目提供的 Z_H 、 Z_{DR} 预测降水量数据。 Z_H 、 Z_{DR} 与降水量之间存在经验估计方法，本文使用**分段拟合方法(PFM)**结合**经验估计公式**建立了三者间的数学关系，并且利用一次降水事件中的多组数据进行拟合参数的优化，求取最优拟合参数。

问题四要求设计数学模型来评估双偏振雷达资料在强对流降水临近预报中的贡献，本质上是变量在模型中的可解释性问题。本文采用**偏最小二乘(PLS)**作为拟合方法提出了一种自适应的变量贡献排序方法，并使用丢弃或加权作为优化数据融合的手段。

关键词：强对流天气 双偏振雷达 ConvLSTM 网络 数据融合

目 录

1.问题重述	4
1.1 问题背景	4
1.2 问题提出	6
2.符号说明	8
3.问题一的模型建立与求解	9
3.1 问题分析	9
3.2 模型建立	10
3.3 算法求解	13
3.4 结果分析	16
4.问题二的模型建立与求解	19
4.1 问题分析	19
4.2 模型建立	19
4.3 算法求解	22
4.4 结果分析	23
5.问题三的模型建立与求解	25
5.1 问题分析	25
5.2 模型建立	25
5.3 算法求解	26
5.4 结果分析	27
6.问题四的模型建立与求解	30
6.1 问题分析	30
6.2 模型建立	30
6.3 算法分析	32
6.4 结果分析	33
7.模型评估与展望	34
7.1.优点	34
7.2 缺点	34
7.3 展望	34

参考文献	35
附录 A:部分预测结果图片	36
附录 B:代码	46

1.问题重述

1.1 问题背景

1.1.1 强降水预测研究背景

强对流天气是指发生突然、变化剧烈、破坏力极大的气候现象，常伴有雷雨、大风、冰雹、飏线、龙卷和短时强降水等灾害性天气。强对流天气容易造成灾害，破坏性比较强。此外，还有局地性较强，空间尺度较小，持续时间或者生命期比较短等特点。这些特性使得强对流天气家族中的每种气候现象都拥有着极强的灾害性。

尺度小又迅速发展的气候现象，在气象学上来讲是预报的难点。在目前的观测条件下，对于强对流天气信息的掌握不够全面，因此对这种天气的认识和了解还存在着一定的局限性。强对流天气的难以预测性是国内外从事气象预测和科学研究的专家、学者的共识^[1]。

现在世界上各个国家对于强对流天气的预报都只是潜势预报，即在某一个相对比较长的时间段里，某个地区可能会出现极端的气候现象。由于时段较长，所以只能给出一个大范围的预报，并不具有特别针对性。随着雷达探测技术和遥感技术的发展，在气象学领域，根据雷达回波响应预测强对流天气的研究日益成熟^[2]。

双偏振雷达变量反映的微物理信息里包含了对流系统的演变状态、空间动力结构等关键信息^[3]，并且可同时发射和接收在水平和垂直两个偏振方向的电磁波，根据两个偏振方向上的回波的强度差别、相位关系等信息获得降水粒子的大小、相态、含水量等信息。双偏振雷达技术在预测强对流天气方面有着巨大的潜力。

综上，应用双偏振雷达数据变量精准的进行强对流预报，仍是目前气象预报的重点难点问题。因此，对双偏振雷达变量的研究，是对强对流天气预报中亟待解决的关键。

1.1.2 双偏振雷达特点及其参量

传统强对流天气临近预报一般是收集和获取气象观测数据，包括地面气象站观测数据、雷达观测数据、卫星云图、上层气象观测等，这些数据提供了关于大气状态的信息。在天气现象发生前的一段时间内，对收集到的观测数据进行质量控制和分析，以了解当前

的天气系统演变和特征，包括对气压、温度、湿度、风向风速等气象要素的分析，对可能产生强对流天气的区域进行预报^[4]。

传统雷达仅能发射和接收一个偏振方向上的电磁波，而新型的双偏振雷达可同时发射和接收在水平和垂直两个偏振方向的电磁波，可以根据两个偏振方向上的回波的强度差别、相位关系等信息获得降水粒子的大小、相态、含水量等信息。近年来，各项研究表明，双偏振雷达变量反映的微物理信息里包含了对流系统的演变状态、空间动力结构等关键信息。根据接收机收到水平和垂直通道的散射信号分别计算出水平反射率因子（ Z_H ），即水平方向的回波强度，单位为 dBZ，主要反映降水的强弱；差分反射率（ Z_{DR} ），即水平和垂直方向回波强度的差异，主要反映了观测区域的降水粒子大小；差分传播相移率（ K_{DP} ），即单位距离上降水粒子导致的水平和垂直方向回波的相位差，主要反映了液态含水量等参量。双偏振雷达响应时空图变化描述了不同时刻和不同地点的气象或目标反射性质的动态演化，这种变化可以由多种因素引起，包括降水、云量、降雪、雷暴活动等气象现象，以及目标的位置和形态。

传统短临预报方法通常存在输入信息不足、模型结构简单等问题，因此难以预报风暴复杂演化过程。近年来，深度学习方法的因可自动有效地从大量观测中训练提取信息，受到大气科学众多研究关注。另一方面，目前对暴雨系统的观测手段逐渐丰富，大量新型观测数据为改进预报带来新的机遇与挑战。

1.1.3 基于双偏振雷达的强降水预测

传统方法根据雷达回波图像预测降水量需要收集大量的降水事件样本，通过气象学专业知识和经验进行估算^[5]。这种方法预测精度较低且容易受到外部条件影响。随着计算机科学与人工智能技术的发展，大量具有时间序列感知能力的深度学习模型被提出，正在逐步取代传统的经验估计方法。

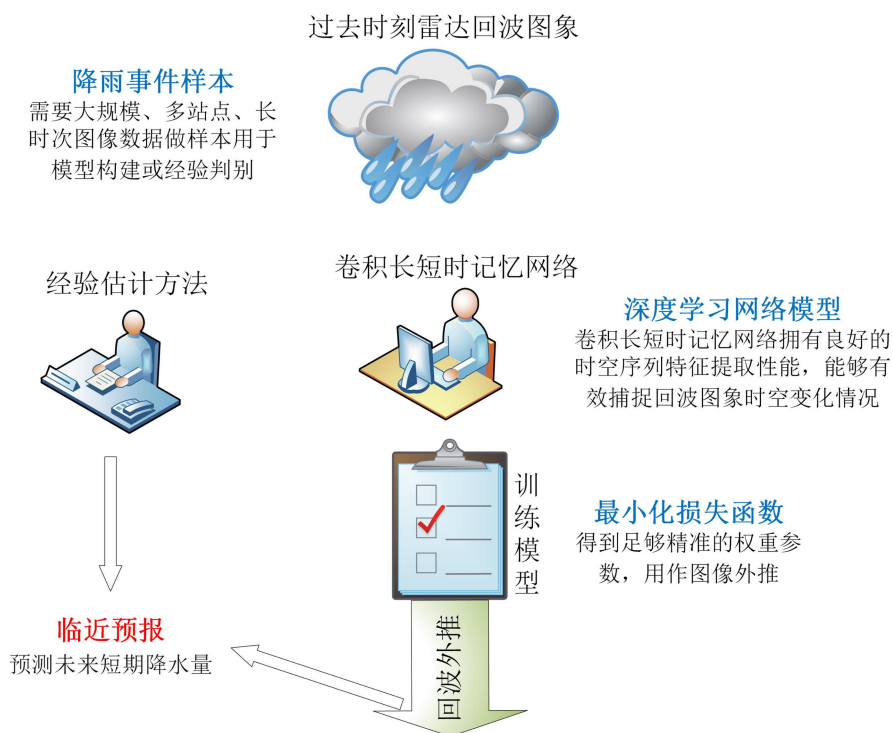


图 1 基于双偏振雷达的降水预测

深度学习模型具有很强的学习能力和表征能力，以多层神经网络为基础，通过学习大量数据中的模式和规律来进行预测，在天气预报中的应用呈现出巨大潜力。通过深度学习模型，可以对复杂的大气系统进行建模和预测，以实现更准确的天气预报。例如，使用卷积神经网络，可以实时检测和跟踪降水带和瞬态天气现象，改善降水的时空预测精度^[6]。

1.2 问题提出

本文主要研究如何有效应用双偏振变量改进对强对流的预报，由以下四个问题组成。

1.2.1 问题一：强对流预报问题

有效应用双偏振变量改进强对流预报，建立可提取用于强对流临近预报双偏振雷达资料中微物理特征信息的数学模型。

1.2.2 问题二：缓解预报模糊效应问题

设计一种数学模型以缓解预报的模糊效应，使预报出的雷达回波细节更充分、更真实，进行强对流预报。

1.2.3 问题三：定量降水估计问题

利用题目提供的 Z_H 、 Z_{DR} 和降水量数据，设计数学模型进行定量降水估计。模型输入为 Z_H 和 Z_{DR} ，输出为降水量。

1.2.4 问题四：评估雷达数据在强对流预报中的贡献，优化数据融合问题

设计数学模型评估双偏振雷达资料在强对流降水临近预报中的贡献，并优化数据融合策略，以便更好地应对突发性和局地性强的强对流天气。

2.符号说明

符号	含义
Z_H	水平反射率因子
Z_{DR}	差分反射率
K_{DP}	比差分相移
i_t	输入门
x_t	输入特征
h_t	短记忆
c_t	长记忆
W	卷积核
f_t	遗忘门
b	偏置参数
σ	Sigmoid 函数
$f(x)$	输入的真实值
y	模型的预测值
m_t	带有动量的梯度值
$beta$	指数衰减率
g	梯度
v_t	学习率权重参数
$variable$	一阶指数平滑值
lr	学习率
θ	角度
a	步长
HSS	海德克技能分数
CSI	关键成功指数
dBZ	雷达回波单位
$loss$	损失率
N	系数
y_i	实际观测值集合 y 中的第 i 个真实观测值
y_i^p	预测值集合中相应预测值的第 i 个观测值
F'	最终的精炼输出
M_c	输入特征图
M_s	空间注意力特征
MLP	多层感知器
$f^{7 \times 7}$	7×7 卷积核的卷积运算

3.问题一的模型建立与求解

3.1 问题分析

对于问题一建立可提取用于强对流临近预报双偏振雷达资料中微物理特征信息的数学模型，临近预报输入为前面一小时（10 帧）的雷达观测量（ Z_H 、 Z_{DR} 、 K_{DP} ），输出为后续一小时（10 帧）的 Z_H 预报。

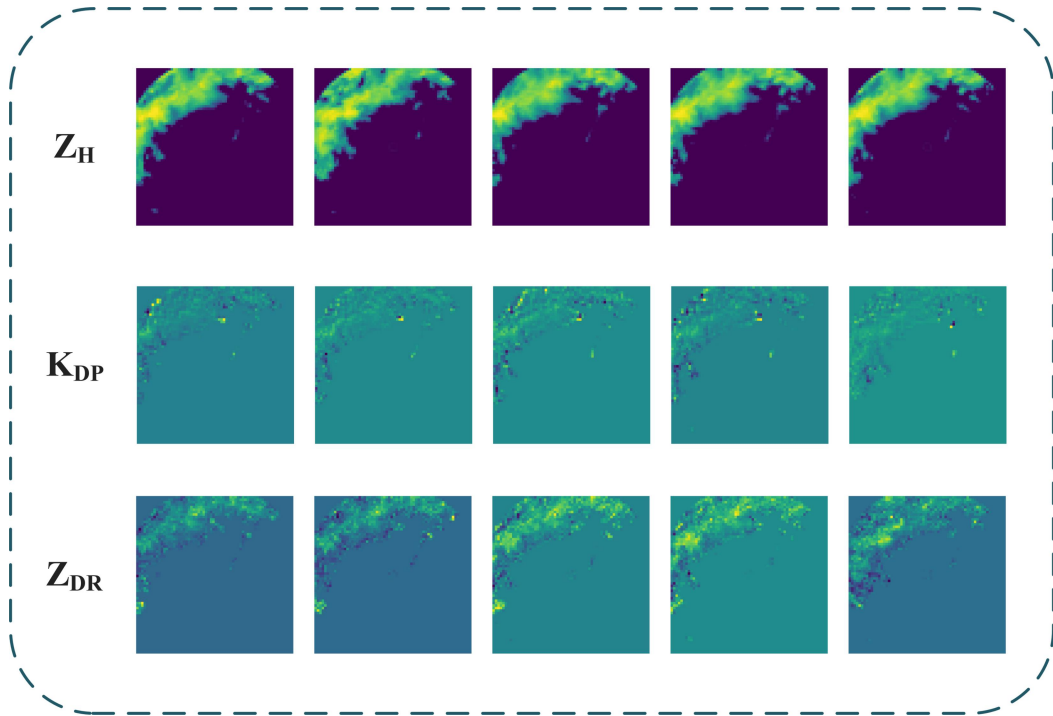


图 2 雷达数据集示意图

图 2 按照时间顺序展示了一系列的雷达回波数据集，其中像素颜色由黄到蓝表示回波由大到小。响应的峰值随着时间的发展在空间域中有明显的移动趋势。

因此，降水预报问题可以被看做时序预测问题，问题给出的自变量是一组具有时序关系的图片，因此考虑使用具有时序感知能力的深度学习方法对问题进行建模。长短期记忆^[7]（Long Short Term Memory, LSTM）是一种特殊的递归神经网络，通过特殊的门控制器设计以学习到数据的长程依赖关系。但单纯的 LSTM 不能很好的感知图片数据空间上的特征信息；卷积长短期记忆^[8]（Convolution Long Short Term Memory, ConvLSTM）是基于 LSTM 的变体结构，其不仅能够建立类似 LSTM 的时序记忆，而且可以拥有类似 CNN 的空间特征提取能力。根据问题一的数据结构，建立了数据融合卷积长短期记忆模型（Data fusion Convolution Long Short Term Memory, DFConvLSTM），对扫描高度为 3.0km 的雷

达回波进行建模。

3.2 模型建立

3.2.1 ConvLSTM 时空序列预测模型

在处理强对流临近预报双偏振雷达资料中微物理特征信息时，需要分析连拍照片类数据，合理运用 t 或 t 之前的输入来预测 $t+n$ 时刻。建立能够感知时间的深度学习网络。LSTM 是一种特殊的循环神经网络 (RNN)，为解决传统 RNN 的梯度消失和梯度爆炸问题，LSTM 引入了门控单元，包括输入门、遗忘门和输出门，以控制信息的流动和记忆的管理。

ConvLSTM 通过引入卷积操作，在获取时空关系上比 LSTM 有更好的效果。其允许网络在时空维度上进行信息传递和特征提取。其不仅可以视频分类，处理遥感图像也能够用于进行气象数据的分析。图 3 所示的是一个 ConvLSTM block 的内部结构。

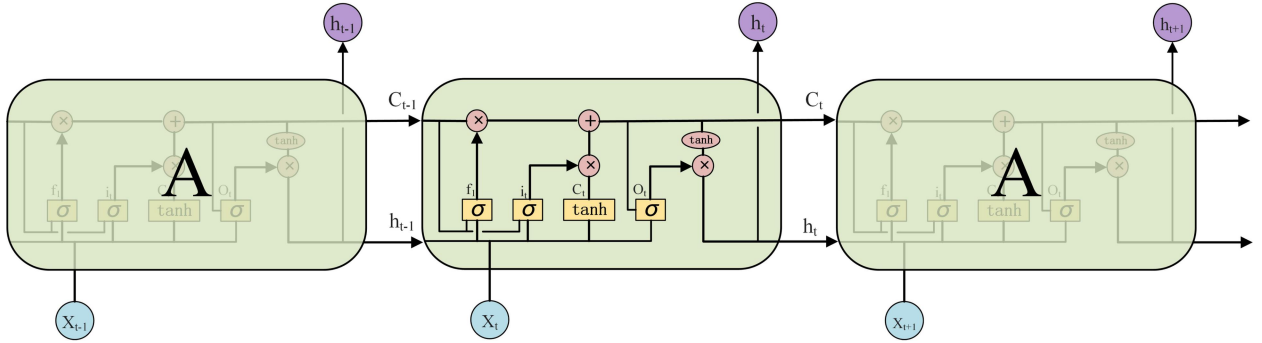


图 3 ConvLSTM block 的内部结构示意图

ConvLSTM（卷积长短时记忆网络）中，有四个关键的组件：输入门（Input Gate）、遗忘门（Forget Gate）、记忆细胞（Cell State）、输出门（Output Gate）。这些组件协同工作，帮助网络处理和记忆时序数据中的信息。

输入门决定了在当前时刻，关键的新信息将被添加到记忆细胞中。输出值范围在 0 到 1 之间，0 表示不添加新信息，1 表示完全添加新信息。其计算通常涉及当前时刻的输入和上一时刻的隐藏状态。输入门公式：

$$i_t = \sigma(W_{xi} * x_t + W_{hi} * h_{t-1} + W_{ci} * c_{t-1} + b_i) \quad (1)$$

遗忘门决定了在当前时刻，关键的旧信息将被保留在记忆细胞中。输出值范围在 0 到 1 之间，0 表示完全忘记旧信息，1 表示完全保留旧信息。其计算通常涉及当前时刻的输入

和上一时刻的隐藏状态。遗忘门公式：

$$f_t = \sigma(W_{xf} * x_t + W_{hf} * h_{t-1} + W_{cf} * c_{t-1} + b_f) \quad (2)$$

记忆细胞是 LSTM 网络中的一个关键组件，它是一个向量，可以保存长期依赖的信息。其值可以根据输入门、遗忘门和当前时刻的输入来更新。它被用在网络的内部存储，保留先前的信息，以便在需要时提供给网络。记忆细胞公式：

$$c_t = f_t * c_{t-1} + i_t * \tanh(W_{xc} * x_t + W_{hc} * h_{t-1} + b_c) \quad (3)$$

输出门决定了在当前时刻，记忆细胞中的关键信息将被输出到网络的下一层或最终输出。其输出值范围在 0 到 1 之间，控制着记忆细胞中的信息被输出的程度。它的计算通常涉及当前时刻的输入和上一时刻的隐藏状态。输出门公式：

$$o_t = \sigma(W_{xo} * x_t + W_{ho} * h_{t-1} + W_{co} * c_t + b_o) \quad (4)$$

其中 σ 是 Sigmoid 激活函数， $*$ 是卷积操作， W 是卷积核， b 是偏置参数。这些门控制了 LSTM 网络中信息的流动和记忆的管理，使得网络能够捕捉并利用长期依赖关系，同时防止梯度消失或梯度爆炸等问题。ConvLSTM 在此基础上引入了卷积操作，以有效地处理时空序列数据，但其核心思想与 LSTM 相似，仍然涉及输入门、遗忘门、记忆细胞和输出门的概念。

3.2.2 多数据融合

双偏振雷达数据中微物理特征信息数据序列具有动态性及不确定性，且经常会有突发事件导致的峰值，因此，结构单一的模型无法充分获取复杂数据的重要特征，预测精度较低。问题一提出使用多数据融合方法来提升预测精度。

在信息融合理论中，按照信息处理的位置可将融合系统分为三个层次，数据集融合，特征集融合和决策集融合。本文所提出的模型使用的信息融合层级为特征及融合；从原始的观测数据中提取重要的时间与空间特征，使用拼接操作进行融合。本文以 ConvLSTM 模型为主干网络，提出了 DFConvLSTM 模型，模型结构如图 4 所示。

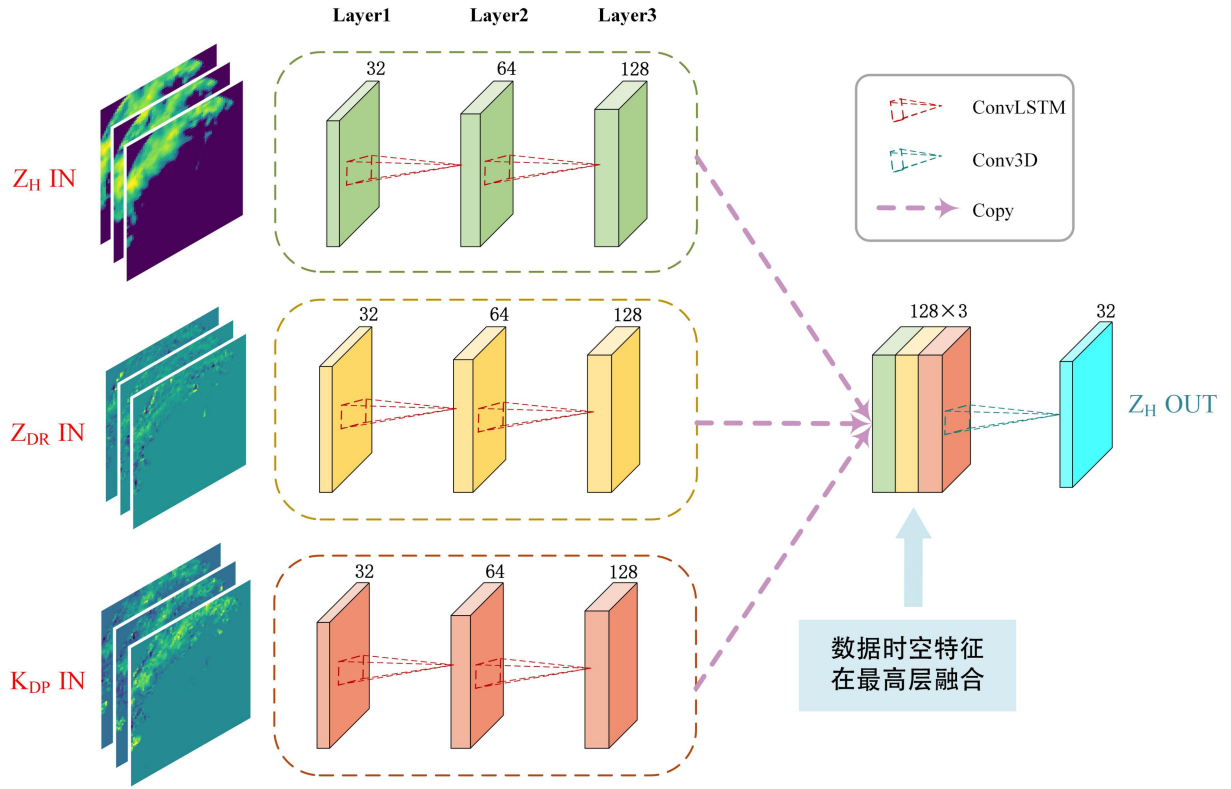


图 4 DFConvLSTM 模型结构示意图

模型的主干由三层 ConvLSTM 网络构成，卷积核个数随着网络加深依次增多为 32，64，128；为实现特征集层次的数据融合设计了并行运行的三通道主干，分别处理数据： Z_H 、 Z_{DR} 、 K_{DP} 。在三层 ConvLSTM 之后将提取到的数据进行拼接操作得到多尺度数据特征图，最后通过一次卷积操作输出预测结果。

3.2.3 损失函数与优化器

损失函数（Loss function）是深度学习模型中的一个重要组成部分，用于衡量预测结果与实际值之间的差异，也是优化算法中评估模型性能的关键指标。MSE（Mean Squared Error，均方误差）是一种常用的损失函数，用于回归问题的模型训练和优化。

MES 的原理是计算预测值与目标值之间的平方差，并取平均值。它衡量了预测结果与真实值之间的平均差异程度，平方的作用是放大差异，使更大的误差对整体损失的贡献更大。MES 损失函数的公式如下：

$$MSE = \frac{1}{n} \sum_{i=1}^n (f(x) - y)^2 \quad (5)$$

其中 y 表示真实值, $f(x)$ 表示模型的预测值, n 表示样本的数量, MSE 损失函数的计算过程如下: ①对于每个样本, 计算真实值与预测值之间的平方差。②将所有样本的平方差相加。③将总和除以样本数量, 得到均方误差。

Adam (Adaptive Moment Estimation) 是一种常用的优化算法, 用于训练深度神经网络。它结合了动量优化和自适应学习率的特性, 旨在加速梯度下降过程, 并且可以适应不同参数的学习率需求。Adam 使用动量来加速收敛。动量可以看作是之前梯度的指数加权平均, 有助于克服梯度下降中的局部最小值问题。它在更新参数时考虑了之前的梯度信息, 使得参数更新更加平稳。Adam 自适应地为每个参数计算不同的学习率。它使用了每个参数的梯度的平方的指数加权平均来调整学习率, 以便更快地更新那些梯度较小的参数, 同时对梯度较大的参数采用较小的学习率, 防止其偏离最优值。Adam 引入了偏差校正机制, 以纠正在训练初期学习率偏高的问题。这是通过将每个参数的学习率初始值除以一个递增的校正项来实现的, 以确保初始更新不会太大。Adam 优化器的公式:

计算历史梯度的一阶指数平滑值, 用于得到带有动量的梯度值:

$$m_t = \beta_{t_1} * m_{t-1} + (1 - \beta_{t_1}) * g \quad (6)$$

计算历史梯度平方的一阶指数平滑值, 用于得到每个权重参数的学习率权重参数:

$$v_t = \beta_{t_2} * v_{t-1} + (1 - \beta_{t_2}) * g^2 \quad (7)$$

计算变量更新值, 由公式 3 可知, 变量更新值正比于历史梯度的一阶指数平滑值, 反比于历史梯度平方的一阶指数平滑值:

$$variable = variable - lr_t * m_t / (\sqrt{v_t} + \epsilon) \quad (8)$$

其中, m_t 是带有动量的梯度值, β_{t_n} 是指数衰减率, lr 是学习率, Adam 优化器根据当前梯度信息以及过去的梯度信息来更新参数, 实现了动量和自适应学习率的组合优势, 使得训练更加高效并且有望更快地收敛到局部最小值。

3.3 算法求解

3.3.1 图片预处理

原图片数据集中图片大小为 $256*256$, 数据量庞大, 使用平均池化方法将其尺寸缩放为 $64*64$ 。同时, 此步骤也能滤除图像中的噪声。 Z_H 、 Z_{DR} 、 K_{DP} 三类数据的单位和数量集

各不相同，使用归一化方法将其缩放到(0,1)区间内。

3.3.2 反向传播流程图

反向传播（Backpropagation）和前向传播（Forward Propagation）是神经网络训练过程中的两个关键步骤，它们共同实现了参数的更新和模型的优化。前向传播是神经网络中的第一步，用于计算模型的预测输出。它从输入数据开始，依次通过每一层的神经元，计算每一层的输出。在每一层中，输入与权重相乘并加上偏置，接着经过激活函数得到该层的输出。前向传播从输入层一直传播到输出层，得到模型的预测结果。

反向传播是神经网络中的第二步，用于计算损失函数对于模型参数的梯度。它通过链式法则来计算梯度的，从输出层开始向后传播，计算每一层的梯度。其目标是找到损失函数相对于每个参数的梯度，以便后续使用梯度下降等优化算法来更新参数。通过反向传播，神经网络能够“反馈”误差，以便对参数进行调整，减小预测误差。

因此，前向传播用于计算模型的输出，而反向传播用于计算模型参数的梯度，以便进行优化。这两个过程一起构成了神经网络的训练过程。在训练过程中，多次执行前向传播和反向传播，不断更新参数，直到损失函数达到满意的值或训练停止条件满足为止。

反向传播的第一步是计算网络的输出与真实标签之间的误差，通常使用损失函数来度量。输出误差表示为对网络输出的导数。第二是计算输出门梯度，输出门的梯度计算是反向传播的关键步骤之一，其梯度会影响到记忆细胞和隐藏状态的梯度计算。梯度的计算通常涉及到输出误差、当前时刻的记忆细胞和隐藏状态。第三步计算记忆细胞的梯度，包括来自输出门梯度和后续时间步的误差传播，其传播的结果用于更新记忆细胞内容。第四步计算遗忘门、输入门和新记忆的梯度，遗忘门和输入门的梯度计算通过记忆细胞梯度和当前时刻的输入计算所得。遗忘门的梯度决定哪些旧信息应该被保留，而输入门的梯度决定哪些新信息应该被添加到记忆细胞。第五计算卷积核的梯度，ConvLSTM 中的卷积核需要计算梯度以更新参数，卷积核的梯度计算涉及到误差传播和卷积操作。最后梯度需要传播回上一层和前一时间步以进一步更新网络参数和损失函数，每一次的循环为一次迭代过程。

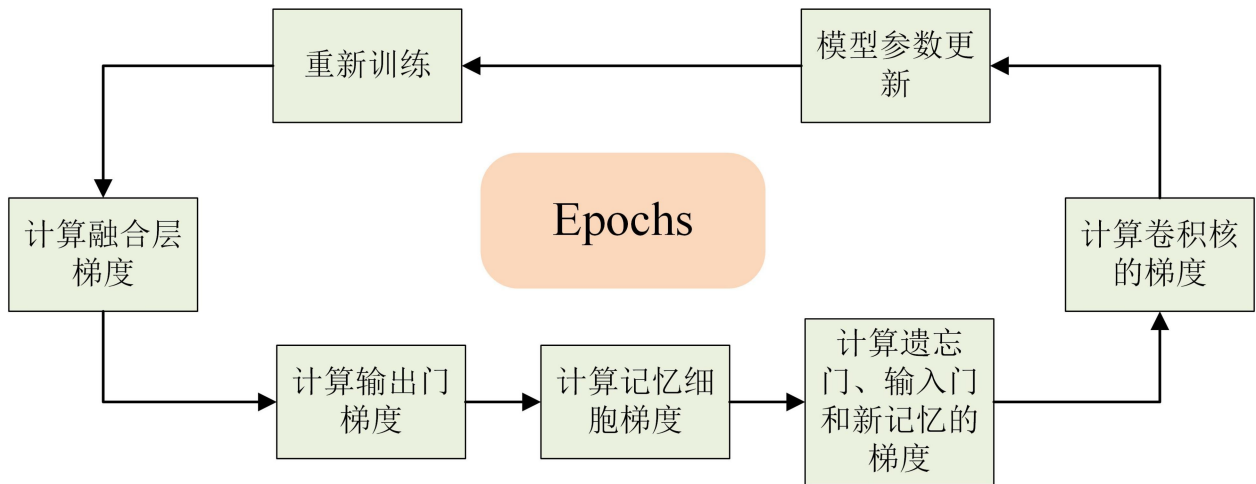


图 5 模型迭代流程

3.3.3 模型调优

学习率 lr (Learning Rate) 是深度学习模型中的一个超参数，用于控制模型参数在每次迭代时更新的步长大小。它是优化算法中的一个重要参数，影响着模型收敛速度和最终收敛到的最优解。 lr 过大可能导致震荡或发散，而 lr 过小可能导致收敛速度缓慢，合理的 lr 能让模型收敛到最小点而非局部最优点或鞍点。在梯度下降的过程中更新权重时的超参数，即以下公式：

$$\theta = \theta - \alpha \frac{\partial}{\partial \theta} J(\theta) \quad (9)$$

其中， α 为更新步长。 lr 越低，损失函数的变化速度就越慢，容易过拟合。虽然使用低 lr 可以确保不会错过任何局部极小值，但也将花费更长的时间来进行收敛，特别是在被困在局部最优点的时候。而学习率过高容易发生梯度爆炸， $loss$ 振动幅度较大，模型难以收敛。

BatchSize 指同时代入训练模型的实例个数，它不仅影响训练速度，也影响模型精度。BatchSize 较小时，抖动大，训练过程有很大运气的成份，可能某个实例将模型带偏了，防止被模型被带偏的方法是使用较小的 lr ，这样即非并行处理，使用较小的 lr ，会使得收敛变慢。

BatchSize 较大时，任务并行执行，训练速度快，且大 Batch 正负样本更均衡可以有效更新，BN 层参数精度更高。代价函数也能相对比较稳定，平滑地下降。但是如果代入了所有数据后再调参，可能会使很多梯度相互抵消，调参也比较粗糙。如果不进行特殊处理，

过大的 BatchSize 将会损失模型精度。另外，较大的 BatchSize 会使模型的泛化能力下降。

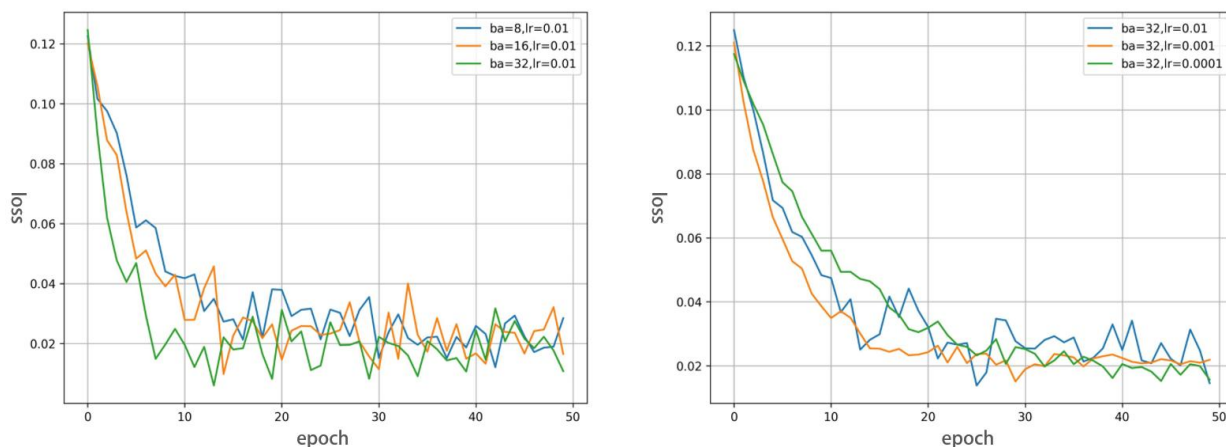


图 6 超参数调优

本文中的所有数据实验在个人计算机（CPU:intel i5-12400f、GPU:GeForce RTX 2060 Super）上使用 Pycharm(Python3.8、架构 TensorFlow-gpu=2.5.0)进行的。通过调用 TensorFlow 中封装的 layers 接口进行 DFConvLSTM 模型的搭建。

重复多组实验得到模型的最优超参数，损失函数下降过程如图 6 所示。为使得损失函数降到最低，提高模型精度，首先固定 lr 为 0.01，设定三组不同的 BatchSize，8，16，32，进行了 50 次的迭代，观察图像可知此三种损失函数在波动上都非常剧烈，当 BatchSize 等于 32 时，损失函数下降幅度最大，故选择 32 为最佳 BatchSize。再设定 BatchSize 为 32 进行第二组实验， lr 设置为 0.01，0.001，0.0001，观察图像显而易见当 lr 为 0.001 时，损失函数下降幅度最大，且波动趋于平缓，最后的损失函数值接近于 0.02。

3.3.4 算法分析

深度学习模型 DFConvLSTM 将 LSTM 的时序理解能力与卷积操作的特征提取能力相结合，同时引入了数据融合方法，从不同类型的时空序列中提取因变量发展趋势的特征。经过训练得到能够拟合 Z_H 序列的最优参数。

3.4 结果分析

3.4.1 评价指标

将生成结果与真实数据比较进行生成结果评价。本文根据海德克技能分数(The Heidk Skill Score, HSS)和关键成功指数(Critical Success Index, CSI)对结果进行评估。首先设定阈值 τ 将 Z_H 变换为二值矩阵, 如果 Z_H 大于 τ , 设置为 1, 否则设置为 0。通过比较原始值和预测值的二值矩阵, 我们得到了 TP 、 FP 、 TN 和 FN , 第一位表示分类器是否预测正确, 第二位表示分类器的预测结果。它们分别表示真阳性、假阳性、真阴性、假阴性四个元素来评价分类器是否预测正确。 HSS 计算方法如下:

$$HSS = \frac{2 * (FN * TN - FN * FP)}{(TP + FN) * (FP + FN) + (TP + FP) * (FP + TN)} \quad (10)$$

CSI 的计算方法如下:

$$CSI = \frac{TP}{TP + FP + FN} \quad (11)$$

本模型分别设定 10、20 和 40dBZ 作为阈值。 HSS 和 CSI 是综合指标, 利用 MSE 来评估性能, 它们同时考虑了检测概率和误报率, 可以直接反映模型的好坏。 HSS 和 CSI 越大, 性能越好。

3.4.2 结果展示及说明

图 7 展示了真实序列 10 帧的演变过程, 图片像素由蓝色, 绿色和黄色逐渐变化, 体现 Z_H 的响应从弱到强的强度变化。在真实序列中, Z_H 的峰值在空间中逐渐向右移动, 在 DFConvLSTM 预测序列中这一特征被很好的理解了。但靠后帧数的生成结果比较模糊, 这可能是模型受到“回归到均值”效应的影响。

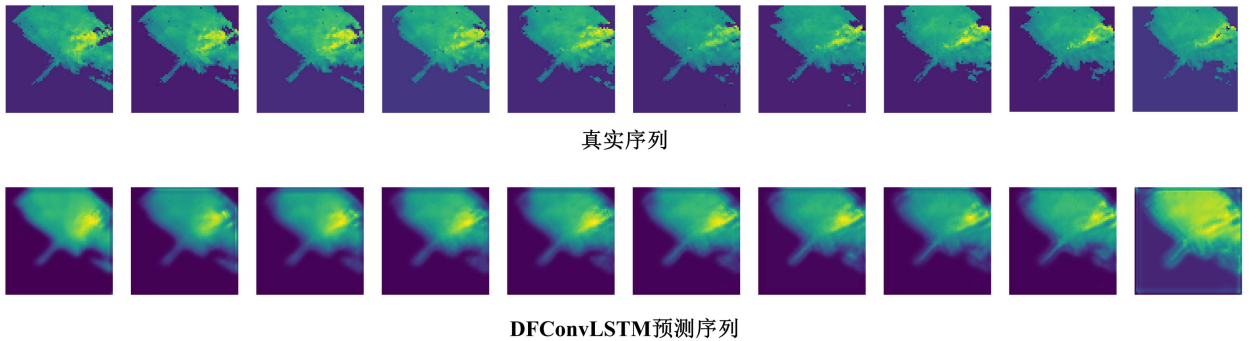


图 7 DFConvLSTM 模型输出结果

选取第五帧生成结果进行分析, 计算两类评价指标结果如表 1 所示。阈值会影响评价指标的大小, 但这种变化是基于评价标准变化的, 实际生成图片的效果不变。

表 1: 单帧图像生成效果

阈值	HSS	CSI
10	0.6815	0.6279
20	0.5926	0.4883
40	0.0215	0.0185
Average	0.4319	0.3782

在阈值为 10 的情况下二者相关性能达到 0.6-0.7 之间。说明此模型能够在实际应用中快速、准确地处理数据，并为决策提供有价值的信息。但是，预测的序列图其 Z_H 响应部分与周围未响应部分边缘轮廓不够清晰锐利，存在一定程度上的模糊感。特征不够清晰明确，这是导致评价指标下降的主要原因。

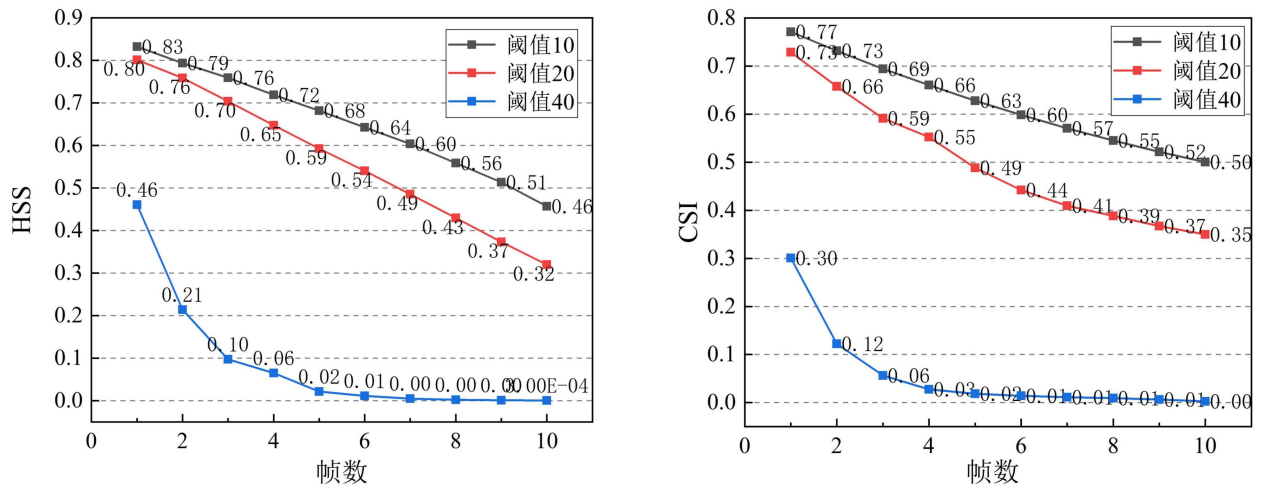


图 8 系列图像生成效果

如图 8 所示，在不同 dBZ 阈值下，每一帧的 HSS 和 CSI 逐渐减小（生成效果逐渐变差），PFST-LSTM 的性能优于所有方法，尤其是阈值在 10 的条件下。在 HSS 方面，由于阈值分别为 10 和 20，我们的方法的预测仅比第二种方法高 0.07% 和 0.96%。当阈值为 40 时，提高到 16.36%。类似的现象也可以在 CSI 中看到。

4.问题二的模型建立与求解

4.1 问题分析

问题二是一个对问题一模型的优化问题，要求缓解预报的模糊效应，使预报出的雷达回波细节更充分、更真实。

问题一中的模型在进行强对流预报的重复预测下，极端观测值趋向于在下一一次测量中向平均值回归（Regression to the mean）的问题，即在进行重复预测时，上一次预测中获得的极高或极低分数会在后一次预测时倾向于向平均值偏移的这种自然倾向。

常规下 ConvLSTM 模型是采用均方差损失函数，MSE 计算真实值和预测值对应点的误差平方之总和，再得到其均值。因此，对模型有不同影响的值，其权重大小一样，而雷达回波数据的回波强度各异，倘若还是一样的权重则会极大影响模型精度和预测效果。因此，本文通过优化损失函数从而改进模型。第二，雷达数据中包含了大量信息，以至于处理和利用这些信息变得非常困难，常常会面临信息过载的问题，难以筛选出有用的信息。因此，本文加入自适应感应机制，目标是优化模型的性能和适应能力。根据环境变化进行适当的调整，以提高模型的效率、准确性和适应性。

4.2 模型建立

4.2.1 损失函数的优化

本文在问题一的模型中采用的是均方差损失函数，而要使预报出的雷达回波细节更充分真实，则需要对损失函数进行优化，从而提高强回波的预测准确率，这也是提高强降水预报效果之关键。为了提高雷达回波预测的准确性，本文引入了一种改进的损失函数，该损失函数由两部分组成：原始均方误差 MSE 损失和加权均方误差损失，它们的权重系数各占 0.5。对于加权均方误差损失函数，需要在计算损失时考虑雷达回波真实值的权重。

通过改进后的损失函数式(12)，引入预测值集合 $\hat{y}$ 和实际观测值集合 y ，其中 N 是系数。新的损失函数在观测值为强回波即 y 值较大时产生更大的影响。其中， y_i 表示实际观测值集合 y 中的第 i 个真实观测值， y_i^p 代表预测值集合 $\hat{y}$ 中相应预测值的第 i 个观测值。基于式

(12)可以推导出式(13)。从式(13)可以看出,改进后的损失函数增加了 $N^2 y_i^2$ 真实值与预测值差异的权重系数。当预测值与真实值存在差异且真实观测值较大时,改进后的损失函数会计算出比改进前更大的数值。

$$loss = 0.5 * loss(\hat{y}, y) + 0.5 * loss(N * y * y, N * \hat{y} * y) \quad (12)$$

$$loss = 0.5 * \frac{1}{n} \sum_{i=1}^n (y_i - y_i^p)^2 + 0.5 * \frac{1}{n} \sum_{i=1}^n N^2 y_i^2 (y_i - y_i^p)^2 \quad (13)$$

这个改进使得强回波的权重增加,从而提升了对强回波的预测效果。权重是根据回波值的大小来分配的,如图 9 所示,对于较大的回波值赋予更高的权重,而对于较小的回波值,赋予更低的权重。这种权重分配策略有助于突出强回波的重要性,从而提高了对它们的准确性预测。

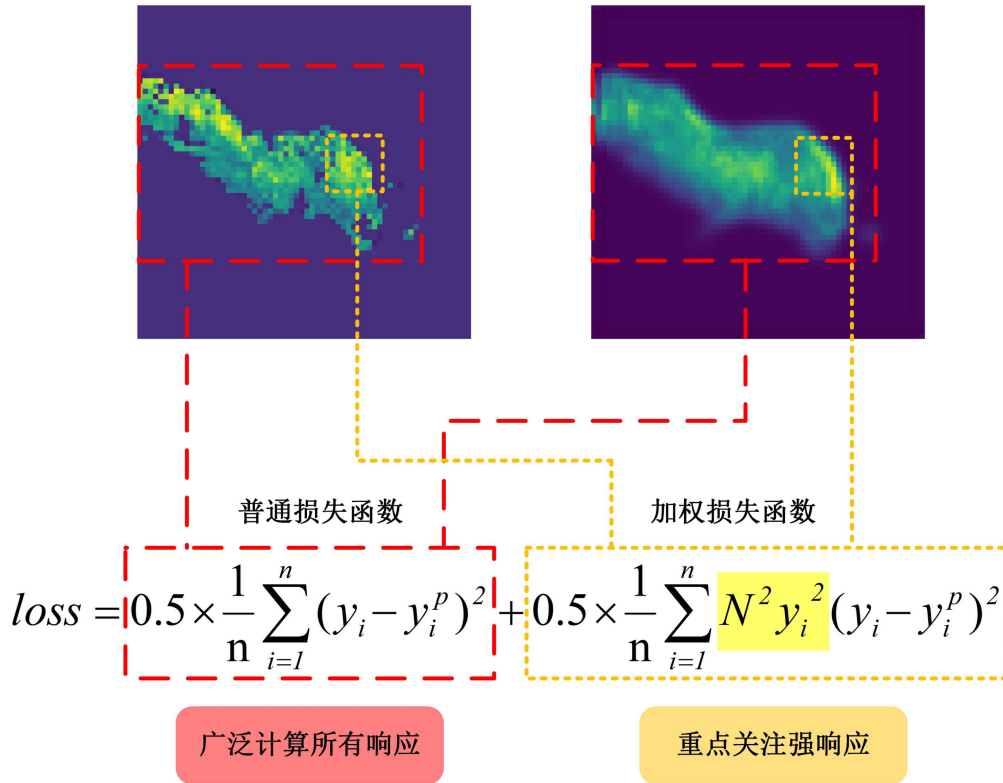


图 9 改进后损失函数特性

4.2.2 CBAM 注意力

CBAM 模块是一种结合通道注意力模块 (Channel Attention Module) 和空间注意力模块 (Spatial Attention Module) 的集成模块,它能够自适应地关注到矩阵中特征点变化较明

显的区域[9]。CBAM 模型结构，与其在主干网络中的位置如图 10 所示。

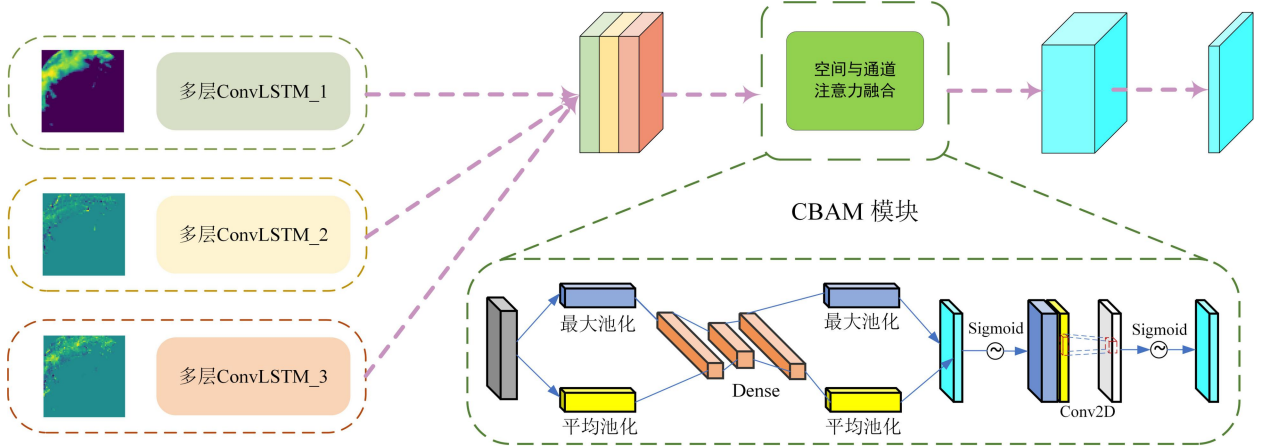


图 10 CBAM 注意力结构以及在主干网络中的位置

相比较单个注意力模块来说，CBAM 模块既能关注卷积网络通道中的注意力信息，也能关注空间中的注意力信息，使经过一系列卷积后的特征能够在 CBAM 模块中得到更多的关注，以致在全连接层能够得到相对应的权重信息，优化分类结果。

通道注意力机制可以自适应地去关注在通道关系中‘哪一些特征更加重要’。模块工作流程如下：首先将经过最大池化层后的特征图 $F (H \times W \times C)$ 通过并行的全局最大池化（Global Max Pooling）和全局平均池化（Global Average Pooling）进行空间维度上的压缩，分别得到两个 $(1 \times 1 \times C)$ 的特征图，再将它们分别输入到两层共享的多层感知机（Multi-Layer Perception, MLP）网络中再次进行维度压缩。在共享 MLP 网络中，一般是由两层全连接层组成，在第一层全连接层中，神经元的个数为 C/r （ r 为减少率），激活函数为 $Relu$ ，在第二层全连接层中，神经元的个数为 C ，随后将两个全连接层输出的特征按元素进行相加，通过最后 $Sigmoid$ 激活函数操作，生成最终的通道注意力特征 Mc 。 Mc 计算公式如式(14)所示。

$$\begin{aligned} Mc(F) &= \sigma(MLP(AvgPool(F)) + MLP(MaxPool(F))) \\ &= \sigma(W_1(W_0(F_{avg}^c)) + W_1(W_0(F_{max}^c))) \end{aligned} \quad (14)$$

式中， σ 为 $Sigmoid$ 激活函数， F_{avg}^c 为经全局平均池化层后的结果， F_{max}^c 为经全局最大池化层后的结果， $W_0 \in \mathbb{R}^{C/r \times C}$ ， $W_1 \in \mathbb{R}^{C \times C/r}$ ， W_0 和 W_1 的权重共享。

空间注意力机制和通道注意力机制类似，但空间注意力机制关注的是在空间关系中‘重要的特征在哪’，当做通道注意力机制的补充。模块工作流程如下：首先将通道注意力的输出特征图 Mc 作为此模型的输入特征图，经过两个通道间的串行的最大池化层和平均池化层，其中经过最大池化层提取通道间的最大特征值，再经过平均池化层提取通道间

的平均值特征，将得到两个尺寸为 $H \times W \times 1$ 的特征图，然后将这两个特征图做通道间的拼接，经过一个 7×7 的卷积核的卷积后，将特征进行降维，降维后的特征大小为 $H \times W \times 1$ ，最后通过 *Sigmoid* 激活函数生成最终的空间注意力特征 Ms 。 Ms 计算公式如式(15)所示。

$$\begin{aligned} Ms(Mc) &= \sigma(f^{7 \times 7}([AvgPool(Mc); MaxPool(Mc)])) \\ &= \sigma(f^{7 \times 7}([Mc_{avg}^s; Mc_{max}^s])) \end{aligned} \quad (15)$$

式中， σ 为 *Sigmoid* 激活函数， $f^{7 \times 7}$ 为 7×7 卷积核的卷积运算， $[]$ 为拼接操作。

4.3 算法求解

4.3.1 损失函数梯度计算

损失函数的梯度计算是指计算损失函数相对于模型参数的梯度。这个过程通过链式法则和反向传播算法实现。①定义损失函数，将其作为衡量模型预测与真实值之间差异的指标。②对于梯度计算，使用固定初始值将模型的参数进行初始化。③通过前向传播，在模型中将输入数据传递到输出层，计算出模型的预测值。④将模型的预测值与真实值进行比较，并计算出损失函数的值，这个值表示模型在当前参数下的预测误差大小。⑤使用反向传播算法，从输出层到输入层，沿着网络的反方向计算梯度。⑥通过链式法则，在每一层计算出相对于损失函数的梯度，将梯度进行传递和累积。⑦使用优化算法，根据损失函数的梯度调整模型的参数，使损失函数的值逐步减小。⑧重复执行步骤③至步骤⑥，持续迭代更新模型的参数，直到达到指定的收敛条件。⑨通过计算损失函数的梯度，进行参数优化，使模型逐渐收敛到较优的状态。优化损失函数的梯度可以通过计算损失函数关于每个参数的偏导数来求得，假设模型产生的预测值 y_i^p 和相应的真实值 y_i ，优化损失函数的定义如式(16)：

$$loss = 0.5 \times (\frac{1}{n} \sum_{i=1}^n (y_i - y_i^p)^2 + \frac{1}{n} \sum_{i=1}^n N^2 y_i^2 (y_i - y_i^p)^2) \quad (16)$$

其中 n 为样本数量。要求得优化损失函数关于预测值 y_i^p 的梯度，则需要计算偏导数（梯度）：

$$\frac{\partial loss}{\partial y_i^p} = 0.5 \times \frac{\partial (\frac{1}{n} \sum_{i=1}^n (y_i - y_i^p)^2 + \frac{1}{n} \sum_{i=1}^n N^2 y_i^2 (y_i - y_i^p)^2)}{\partial y_i^p} \quad (17)$$

通过链式法则，在每一层计算出相对于损失函数的梯度，将梯度进行传递和累积，将其展开为：

$$\frac{\partial loss}{\partial y_i^p} = \frac{1}{2n} \frac{\partial (\sum_{i=1}^n (y_i - y_i^p)^2 + \sum_{i=1}^n N^2 y_i^2 (y_i - y_i^p)^2)}{\partial y_i^p} \quad (18)$$

接下来，以单个样本为例计算梯度，其展开式如下：

$$\frac{\partial (y_i - y_i^p)^2}{\partial y_i^p} = 2(y_i^p - y_i) \quad (19)$$

因此，优化损失函数对于预测值 y_i^p 的梯度为：

$$\frac{\partial loss}{\partial y_i^p} = \frac{1}{n} (\sum_{i=1}^n (y_i^p - y_i) + \sum_{i=1}^n N^2 y_i^2 (y_i^p - y_i)) \quad (20)$$

4.3.2 算法分析

问题二根据问题一的模型进行了两方面改进，其一是损失函数改进，使得模型对雷达回波的强响应更加敏感。其二是增加了 CBAM 注意力机制，对融合特征图中的重要信息从两方面进行了关注。相比于为优化前的 DFConvLSTM 模型，优化后的模型具有更强的泛化能力，并且缓解“回归到均值”效应给模型带来的不利影响。

4.4 结果分析

4.4.1 结果展示说明

针对问题二，本文优化了激活函数，使得原始均方误差 MSE 损失和加权均方误差损失的权重系数各占 0.5；加入了卷积块注意模块(CBAM)，优化后的 DFConvLSTM 的融合模型预测出来后续一小时（10 帧）的 Z_H 预报如图 11 所示。显然，问题二的预测结果的响应与真实序列依然保持一致；在评价指标上，优化后的预测结果和原始序列在相关性上全面提高，特别是在阈值为 10 的条件下，CSI 系数在 0.67 左右，HSS 系数能达到 0.75 左右。

表 2：单帧图像生成效果

HSS	CSI
-----	-----

阈值	优化前	优化后	优化前	优化后
10	0.6815	0.7484	0.6279	0.6728
20	0.5926	0.6835	0.4883	0.5649
40	0.0215	0.0803	0.0185	0.1066
average	0.4319	0.5040	0.3782	0.4481

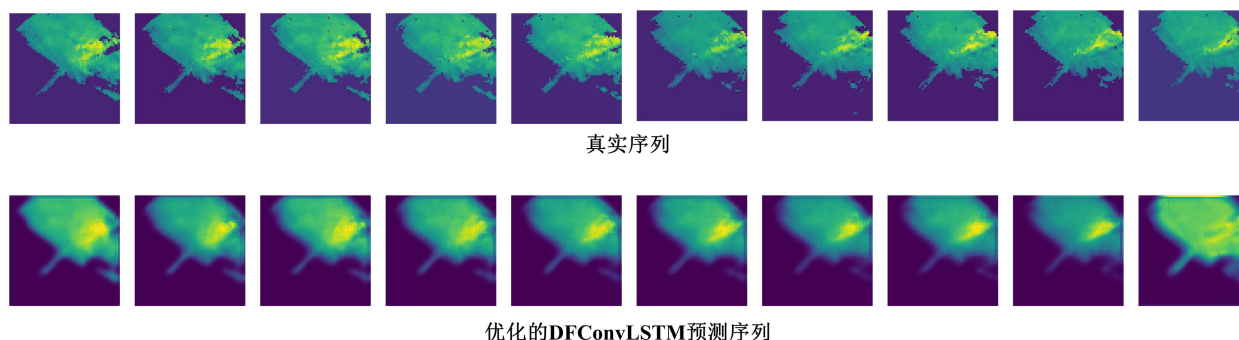


图 11 优化的 DFConvLSTM 模型输出结果

4.4.2 对比问题一效果

显然，问题二优化后的模型预测结果相对于问题一的结果在响应与未响应部分，边缘轮廓更加清晰锐利，模糊感有所下降；另外，不同阈值下的 HSS 和 CSI 两种综合指标均大幅提高，相关性增高。预测结果与真实值非常接近，这证明本模型能够准确地捕捉到数据中的模式和趋势，稳定地实现高质量的预测，并展现出在广泛的数据范围内的优异表现。

5.问题三的模型建立与求解

5.1 问题分析

问题三提出利用 Z_H 及 Z_{DR} 建立数学模型进行定量降水估计,要求模型输入为 Z_H 和 Z_{DR} , 输出为降水量。这类问题可抽象为二元变量的拟合问题,将 Z_H 图片中的像素展平为一维自变量 x , 同理,将 Z_{DR} 图片中的像素展平为一维自变量 y 。建立拟合模型以预测未知的降水量。观察自变量与因变量的特征:数据具有明显的非线性关系,重要信息呈现分段式分布特征。通过查阅文献可知,雷达反射率和降水之间的经验性关系,通常表述为 $R=a*z^b$ 其中 R 为降水量, z 为雷达反射率, a 和 b 为经验性参数,通常在不同地区及不同降水类型下有差异。

综合上述两点,使用分段拟合方法对问题三建立数学模型。

5.2 模型建立

5.2.1 基拟合函数

本文采用降水强度分段拟合方法(Piecewise fitting method, PFM)来选择合适的降水公式进行拟合。降水强度 R 的单位是 mm/h, Z_H 的单位是 mm^6m^{-3} , Z_{DR} 的单位是 dB, K_{DP} 的单位是 $^\circ/\text{km}$ 。在降水粒子形态接近于球形时,即 Z_{DR} 的值接近于 0dB 时, Z_{DR} 的指数 c 往往为负值,这会导致降水估测误差较大。因此,本文采用特定的降水估测拟合形式。对于小雨情况下 Z_{DR} 和 K_{DP} 的值都非常接近零,信噪比(SNR)较小,观测误差较大,所以选择式(21)进行拟合;当大雨时, Z_H 和 Z_{DR} 受到衰减的影响,回波强度较大,观测误差也较大,因此选择式(22)进行拟合,可以取得更好的效果。

$$R_1(Z_H) = aZ_H^b \quad (21)$$

$$R_2(Z_H, Z_{DR}) = aZ_H^b Z_{DR}^c \quad (22)$$

5.2.2 目标函数

$$\text{Min} \sum_{b=1}^B \theta(\hat{R}_i, R_i) \quad (23)$$

以最小化预测降水量和真实降水量间误差 θ 为目标，进行参数拟合。

5.2.3 约束条件

①断点约束：根据降水量的大小定义每个区域分段点的位置。

②边界约束：对拟合结果参数进行边界约束，以保证其在自然条件下的合理性。

5.3 算法求解

5.3.1 算法简化

上述工作建立的模型是一个分段非线性拟合模型。为方便求解将其转化为分段线性拟合问题，对基础拟合函数 $R_1 R_2$ 求对数得到下式：

$$\ln R_1(Z_H) = \ln a + b \ln Z_H \quad (24)$$

$$\ln R_2(Z_H, Z_{DR}) = \ln a + b \ln Z_H + c \ln Z_{DR} \quad (25)$$

将问题转变为线性拟合模型和二元线性拟合模型。

5.3.2 参数优化

条件给出了降水事件中按时序排列的多组数据，对每一组数据的模型求解得到的经验拟合参数不同。为得到某次降水时间的最佳拟合参数，本文提出根据评价指标 MSE 对经验参数进行优化。最小二乘法是一种简单高效的优化技术，它通过最小化误差的平方和寻找数据的最佳函数匹配，并使得这些求得的数据与实际数据之间误差的平方和为最小。

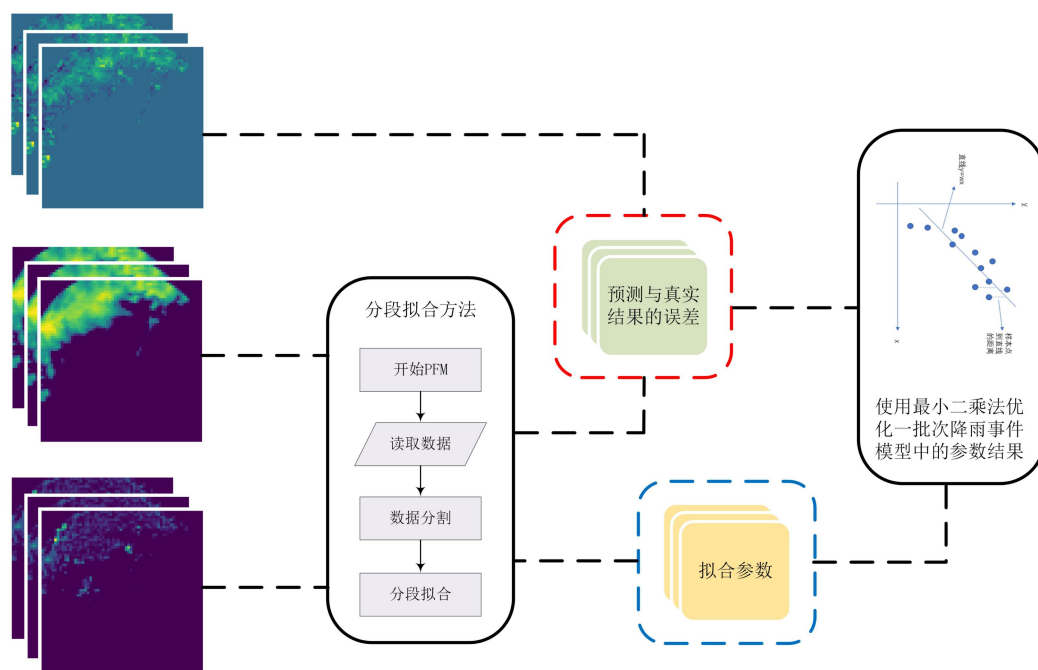


图 12 问题三建模流程

问题三模型的整套流程如图 12 所示，首先使用分段拟合法求得线性拟合的经验参数，在一次降水事件中存在多次拟合过程，求各组预测值与真实值之间误差，并将其送入最小二乘优化模型中得到经验参数的最优组合。

5.3.3 算法分析

问题三的模型需要依赖一定的先验知识。1.不同降水量需要用不同的公式进行拟合。2. Z_H 和 Z_{DR} 与降水量 R 之间的关系可以转换为求对数后的多元线性回归方程。基于以上先验知识使用分段拟合法建立了三者间的数学关系。该算法简单有效，但缺乏泛化能力，可能导致实际预测的精度不足。

5.4 结果分析

5.4.1 评价指标

为了评估拟合的降水估测公式的实际效果，选取相关系数 (Correlation coefficient, CC)、均方根误差 (Root mean square error, RMSE)，作为评估指标，对利用拟合的公式估测的降水强度进行评估。

$$CC = \frac{\sum_{i=1}^n (R_i - \overline{R_i}) \left(\hat{R}_i - \overline{\hat{R}_i} \right)}{\sqrt{\sum_{i=1}^n (R_i - \overline{R_i})^2 \sum_{i=1}^n \left(\hat{R}_i - \overline{\hat{R}_i} \right)^2}} \quad (26)$$

$$RMSE = \sqrt{\frac{\sum_{i=1}^n \left(R_i - \hat{R}_i \right)^2}{n}} \quad (27)$$

上述方程中， R_i 代表降水估测公式计算得到的降水强度， $\hat{R}_i$ 用来表示通过雨滴谱观测数据计算得出的降水强度。上划线标记的变量表示对它们求平均值，而 n 表示评估所使用的数据样本对的数量。

5.4.2 结果展示说明

表 3 所示的是随机抽取 4 次降水事件进行拟合后得到的经验参数。在不同降水事件下拟合出的参数是不同的，但只在某个区间内波动，这点也契合模型建立过程中边界约束的思想。在同一次降水事件中， $R_1(Z_H)$ 的系数 a 总是大于 $R_2(Z_H, Z_{DR})$ 的系数 a ； $R_1(Z_H)$ 的系数 b 总是小于 $R_2(Z_H, Z_{DR})$ 的系数 b 。这一现象说明；在使用 $R_1(Z_H)$ 公式拟合时偏置项对结果影响较大，在使用 $R_2(Z_H, Z_{DR})$ 公式拟合时系数项对结果的影响较大。同时，系数 c 在预测模型中多为负数，说明 Z_{DR} 和降水量呈现非线性的负相关关系。

表 3 不同降雨事件对应的拟合参数

降水事件	公式形式	偏置 a	系数 b	系数 c
NO.001	$R_1(Z_H)$	0.1542	0.4277	×
	$R_2(Z_H, Z_{DR})$	0.0325	0.6384	-0.0894
NO.002	$R_1(Z_H)$	0.0933	0.5131	×
	$R_2(Z_H, Z_{DR})$	0.0198	0.7658	-0.2461
NO.003	$R_1(Z_H)$	0.0611	0.5756	×
	$R_2(Z_H, Z_{DR})$	0.0124	0.8413	-0.3318
NO.004	$R_1(Z_H)$	0.0201	0.6663	×

$R_2(Z_H, Z_{DR})$	0.0069	0.9376	-0.4007
--------------------	--------	--------	---------

表 4 所示的是随机抽取 4 次降水事件的拟合效果。其中各次降水预测结果与真实结果之间的相关系数都接近 1，可以认为两组数据强相关；从预测结果上来看，本节提出的模型预测结果误差在可接受的范围内。

表 4：不同降雨事件的拟合效果

降水事件	平均雨强	CC	RMSE
NO.001	6.22	0.9945	1.2367
NO.002	7.68	0.9931	1.4565
NO.003	9.35	0.9904	1.9209
NO.004	12.08	0.9886	2.6807

6.问题四的模型建立与求解

6.1 问题分析

问题四提出设计数学模型来评估双偏振雷达资料在强对流降水临近预报中的贡献，并优化数据融合策略，并优化数据融合策略，以便更好地应对突发性和局地性强的强对流天气。

在第一问，本文已经阐述了一种输出为后续一小时（10 帧）的 Z_H 预报的模型建立方法，使用等高面为 3km 的双偏振雷达响应数据预测后续响应，从而达到降雨预测的目的，其核心思想是建立具有时空感知能力的深度学习模型。除此之外，题目还给出了三个等高面 1km、3km、7km 的双偏振雷达响应数据。这些数据同样包含能够预测降雨量的信息，但同时也包含大量的无信息变量或共线变量。

机器学习或深度学习方法能够自适应地提取多维数据中的关键信息；对预测结果起关键作用的变量在模型中被赋予较大的权重。然而受限于深度学习模型的“黑箱结构”，模型的解释性分析较为复杂。

根据上述分析，对融合后的雷达响应数据建立浅层的机器学习模型，通过基于学习的方式了解各项雷达响应在强对流降水临近预报中的贡献，对模型的输入变量进行人工干预，以实现更高的预测精度。

6.2 模型建立

6.2.1 数据集构建

自变量数据集 X 由 10 帧 3 组高度的双偏振雷达响应数据构成，数据集 Y 由一张区域降雨图构成， X 矩阵的数据量大小为 $(64, 64, 90)$ ，其数值依次代表样图片长度、图片宽度、采样帧数，将数据进行重构得到 $(4096, 90)$ ，双偏振雷达响应的每个响应点都对应 Y 区域的一个降雨数据，共 4096 个样本，90 个维度。

6.2.2 偏最小二乘

PLS 回归旨在多元自变量 X 和 Y 之间形成回归模型。

$$Y = TQ^T + F, X = TP^T + E \quad (28)$$

其中 $T \in \mathbf{R}^{n \times r}$ 是由观测值的线性组合组成的 r 个主分量, $P \in \mathbf{R}^{p \times r}$ 和 $Q \in \mathbf{R}^{1 \times r}$ 是 r 个主分量的系数矩阵, $E \in \mathbf{R}^{n \times p}$ 和 $F \in \mathbf{R}^{n \times 1}$ 是残差矩阵。在方程 (5) 的条件下找到一组向量 $W=(w_1, w_2, \dots, w_r)$, 此过程遵循以下优化标准:

$$\begin{cases} \hat{w}_k = \arg \max_w w^T X^T Y Y^T X w \\ \text{s.t. } w^T w = 1, w^T S_{XX} \hat{w}_j = 0, j = 0, 1, \dots, r-1 \end{cases} \quad (29)$$

其中 w 是 X 中每个谱行的线性组合系数, S_{XX} 是 X 的样本方差。上述算法通过降低自变量的维数来捕获响应的最大公方差, 从而提高高维数据的预测性能。通过多次迭代获得最佳因子数并建立模型。最终模型可以概括为等式 (30)。

$$Y = [1 \ X] \begin{bmatrix} b_0 \\ b \end{bmatrix} + e = \hat{Y} + e \quad (30)$$

其中 $X (m \times n)$ 表示 m 个样本, 每个样本具有 n 个特征, Y 和 $\hat{Y}$ 分别是实际值和预测值, e 是实际值与预测值之间的偏差, b_0 和 b 是偏差和系数。在主成分提取过程中, PLS 会注意提取的分量对因变量的影响。因此, PLS 在考虑自变量对因变量的决定性的同时, 在很大程度上保留了数据的有用信息。

6.2.3 算法流程图

变量的贡献评估流程如图 13 所示。通过 N 次采样运行, 以迭代的方式选择 N 个变量子集, 最终选择 RMSECV 值最低的子集作为最优子集。当 $i \leq N$ 时, 开始采样, 随机选择 k 个样本(X^k, y^k), 使用 V_{sel_old} 建立 PLS 模型。记录绝对回归系数 b_j , 有 $w_j = b_j / \sum(b_j)$, 使用 $r_i = a e^{-k_i}$ 计算变量的比率。选取保留的 $p \times r_i$ 变量的子集, 用 V_{sel_old} 表示, 用 V_{sel_new} 计算 RMSECV, 经过 N 次采样运行得到 N 个变量子集和相应的 N 的 RMSECV 值。最后, 选择 RMSECV 最低的子集作为变量的最佳子集。也就是对降雨量预测贡献最大的几个变量(雷达回波响应)。

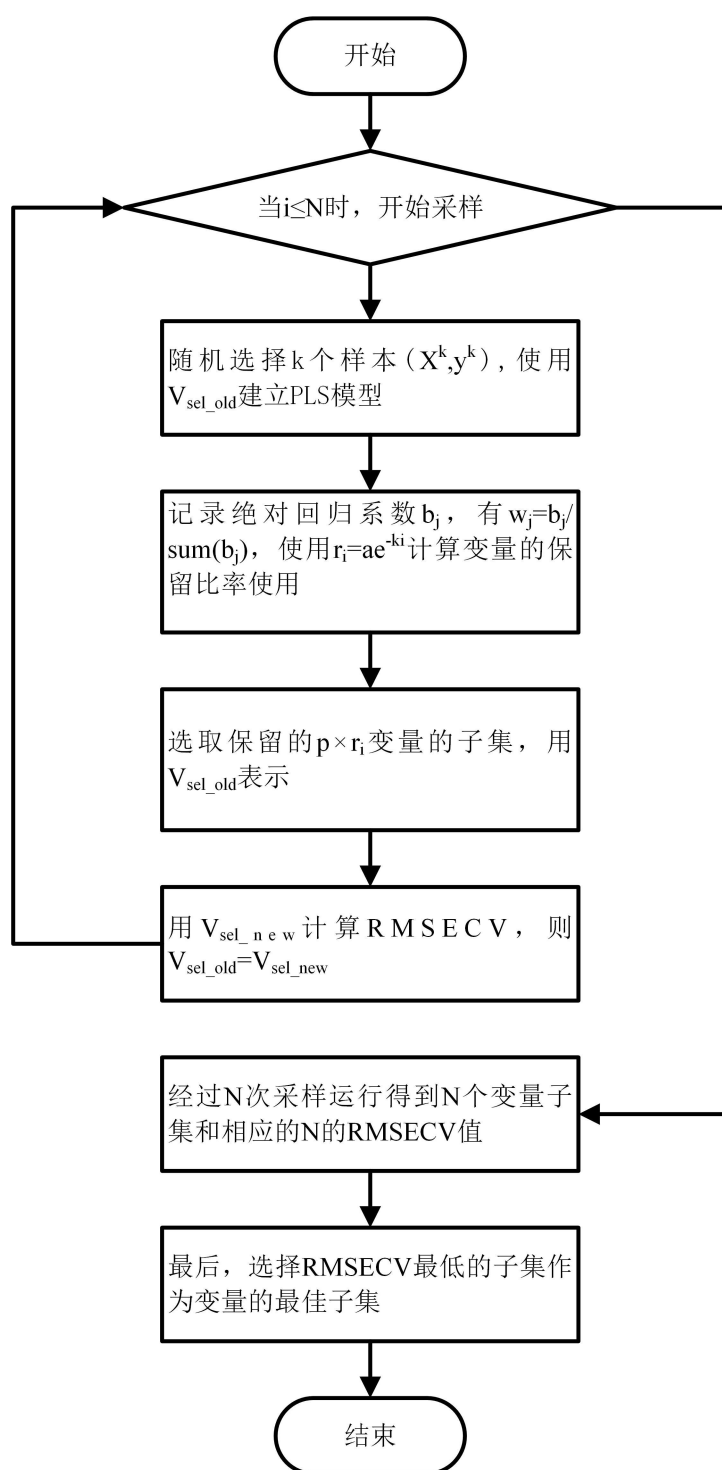


图 13 变量贡献分析流程

6.3 算法分析

本节继承第三问线性拟合方法的思想，同时将问题四进行了抽象与简化；即针对一次降雨事件，降水量的预测问题可以转变为一个多元线性回归问题。其中评价结果最优的一

组变特征子集即是对降雨预测贡献最大的一组雷达回波响应。本文还设计了一种算法流程以自适应的找到这一类特征子集。

6.4 结果分析

对数据集中的 100 次降雨事件进行了分析，在得出的结果中， K_{DP} 或第 10 帧图片在最优变量的筛选结果中频次较高，这说明 K_{DP} 变量对降水量的估计起到了关键的作用，且时间靠后的数据帧更具有代表性。

表 5：100 次降雨事件分析结果

	Z_H_1	K_{DP_1}	Z_{DR_1}	Z_H_3	K_{DP_3}	Z_{DR_3}	Z_H_7	K_{DP_7}	Z_{DR_7}
1	51	57	53	52	56	48	47	54	42
2	60	64	59	57	60	51	52	58	45
3	65	68	61	61	66	56	61	61	50
4	69	72	67	65	69	63	66	63	56
5	72	74	70	70	71	65	70	67	61
6	78	80	77	76	77	72	73	75	63
7	82	86	83	80	84	80	77	82	71
8	86	88	85	83	86	82	80	83	77
9	90	91	90	87	90	88	84	88	80
10	93	96	94	92	95	91	88	93	87

7.模型评估与展望

7.1.优点

- 1.使用数据融合方法增强了模型的泛化能力。
- 2.深度学习模型可以自动捕捉数据的时空特征，实现端到端的输出。
- 3.经验估计方法通常不需要复杂的数学模型或假设，简单易用，在需要快速决策或预测的情况下非常有用。

7.2 缺点

- 1.深度学习模型计算量大，运算成本高，不利于进行强对流天气的实时预测。
- 2.偏最小二乘方法无法理解数据的时序特征，这可能会导致对变量贡献程度的误判。

7.3 展望

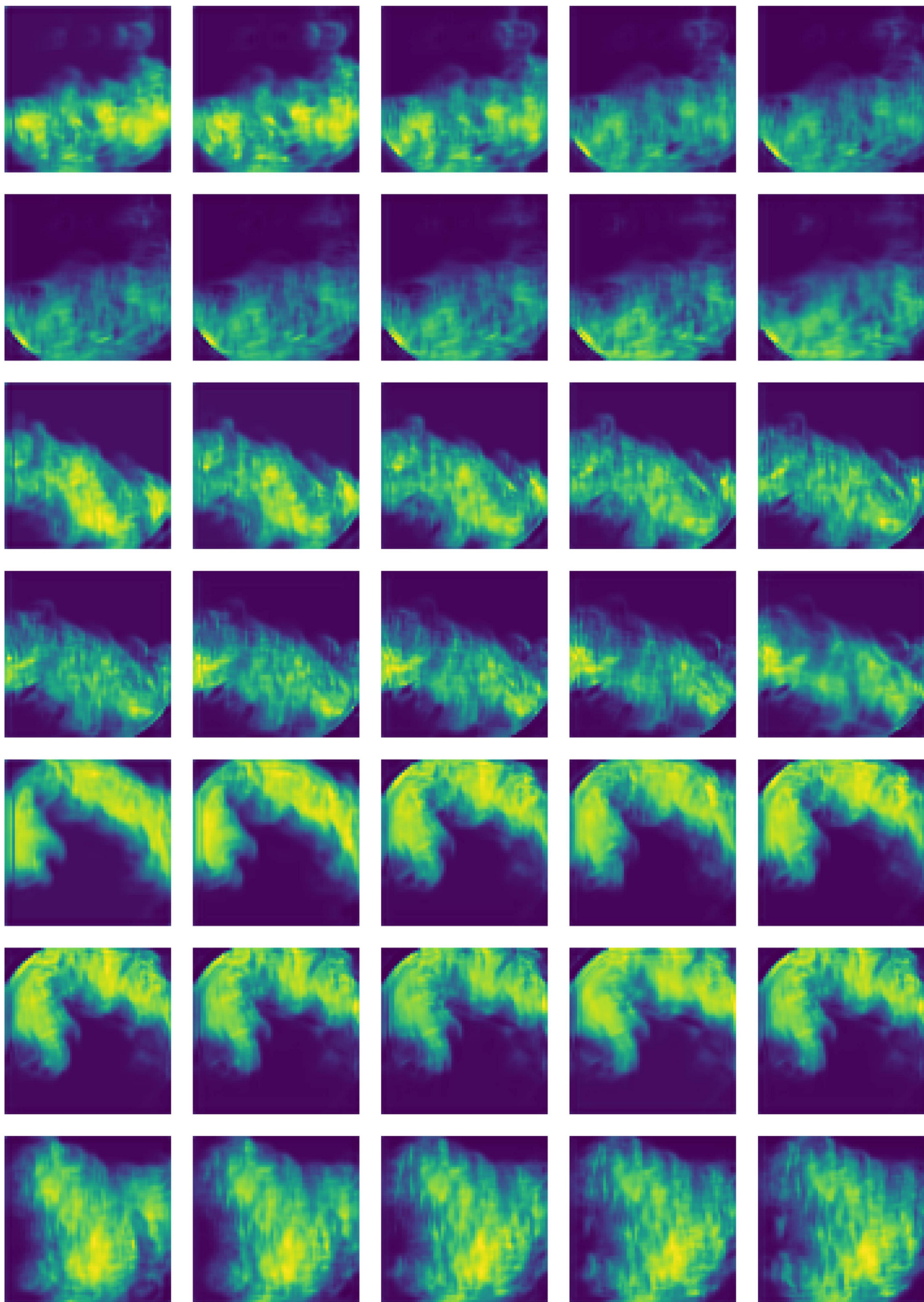
在建模的初期，数据预处理和数据清洗的程度不够，只是简单的使用了平均池化和归一化的预处理操作；雷达回波图片中应该存在异常样本没有被考虑到，平均池化方法能够平滑数据，但也存在使图片模糊的风险。

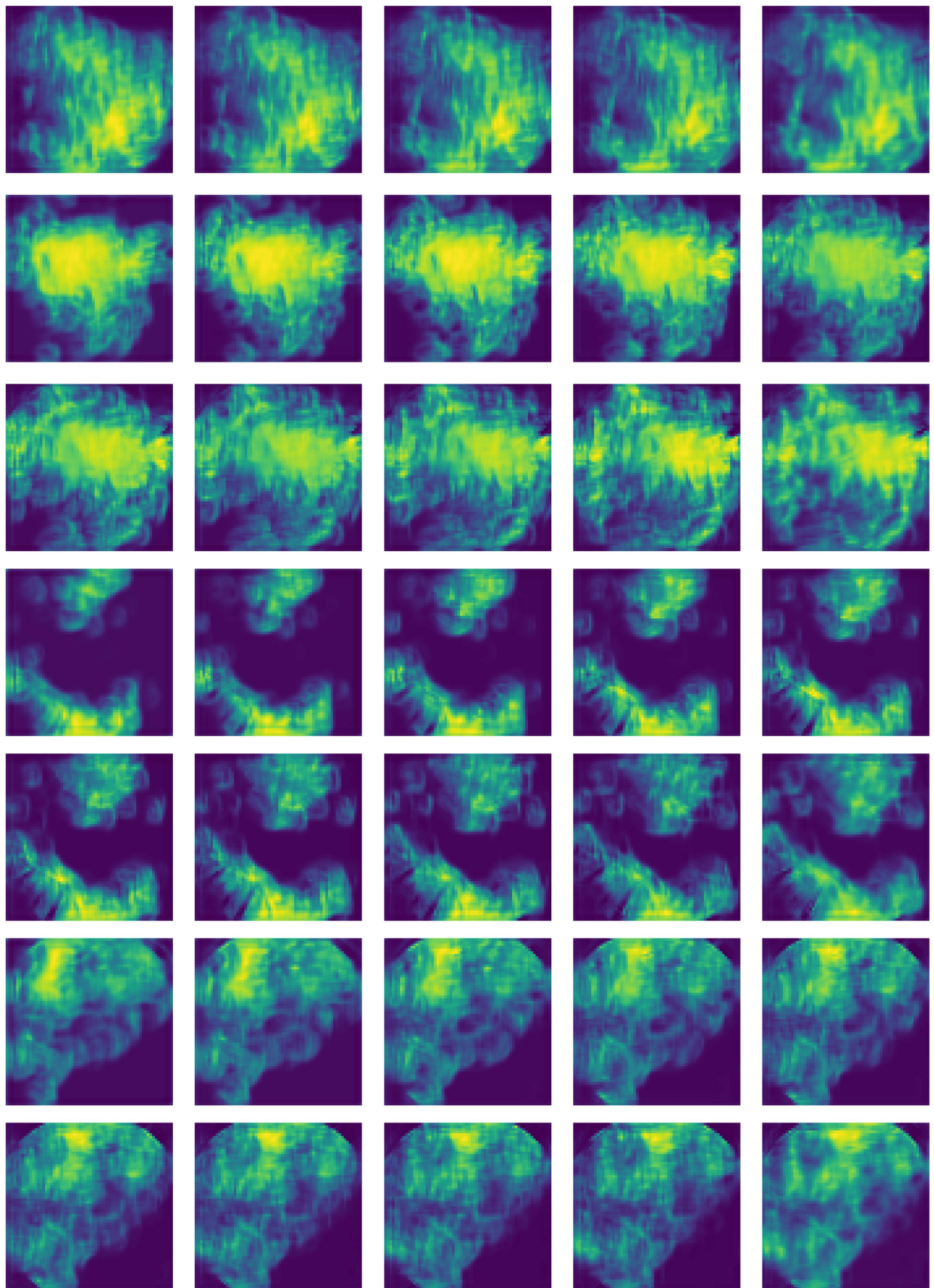
本文的所有模型都建立在题目所给的 258 次降水事件数据中，通过查阅文献可知，强对流天气还包括：冰雹、飑线、龙卷等，对于这些现象应该进行更加深入的数学模型研究。

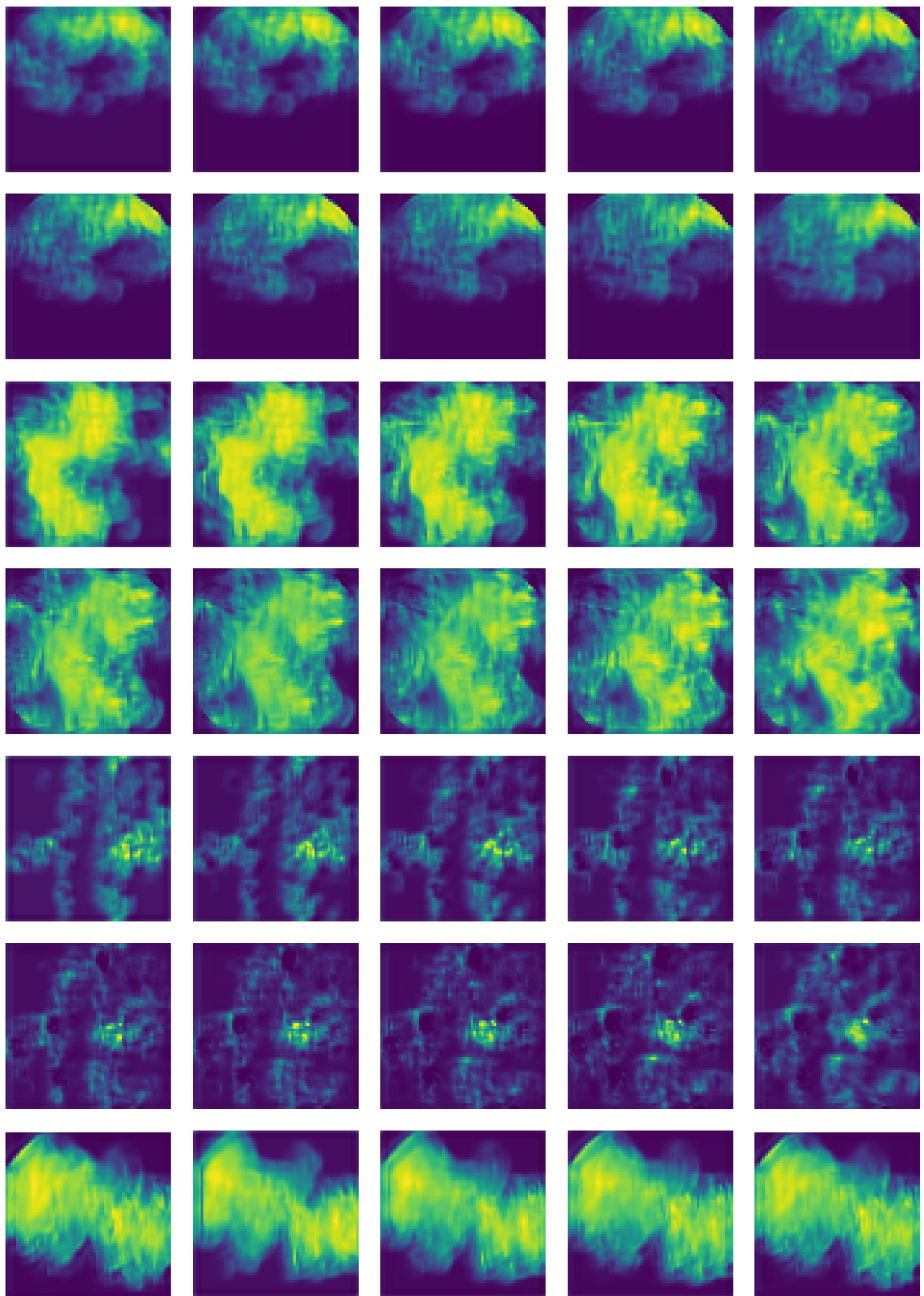
参考文献

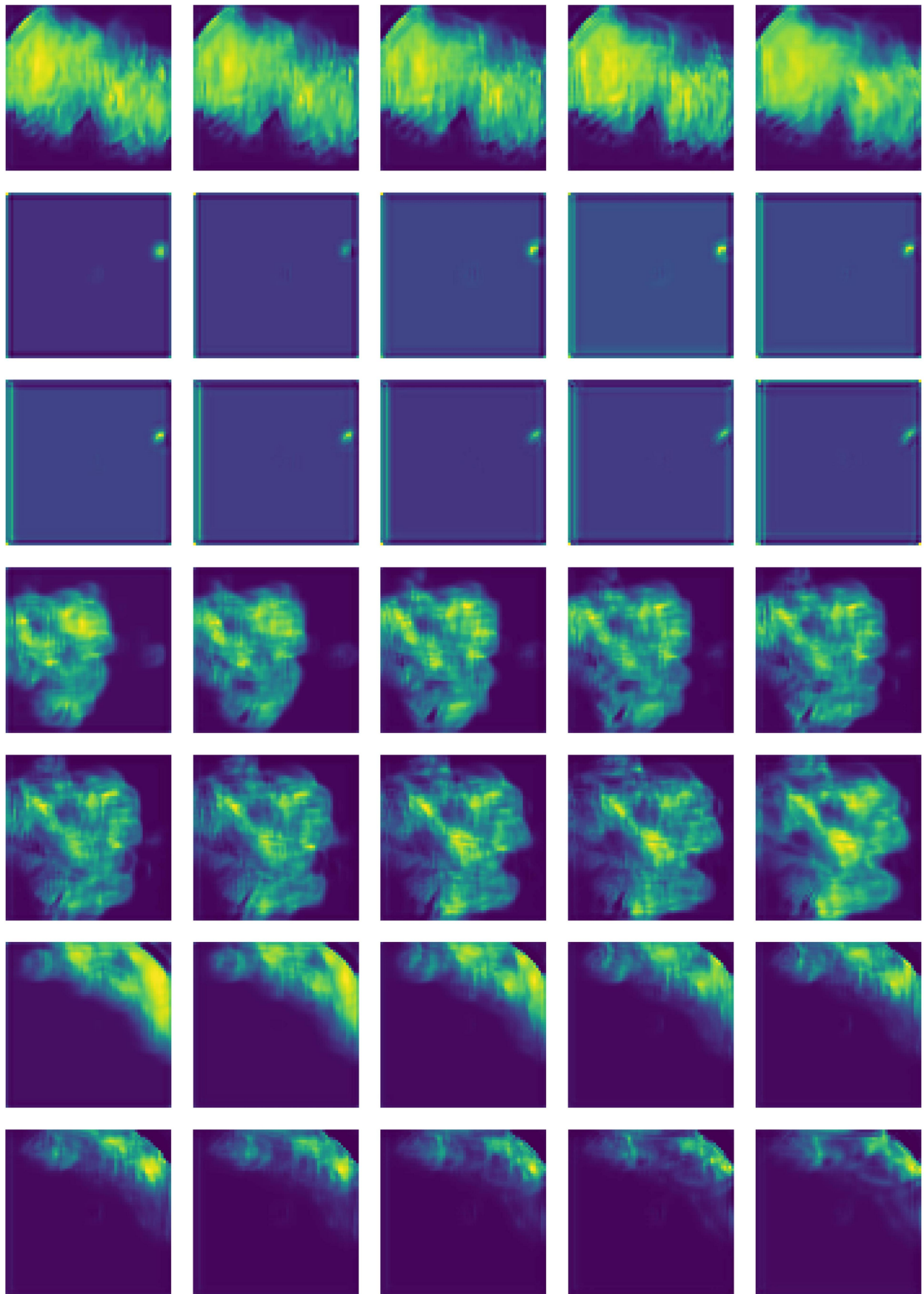
- [1] 曲明星 and 唐冰然, "探究如何做好强对流天气的气象预报工作," *农业灾害研究*, vol. 11, no. 07, pp. 111-112, 2021.
- [2] 谷军霞, 师春香, and 潘旸, "天气雷达定量估测降水研究进展," *气象科技进展*, vol. 8, no. 01, pp. 71-78, 2018.
- [3] "Improving Nowcasting of Convective Development by Incorporating Polarimetric Radar Variables Into a Deep-Learning Model," *Geophysical Research Letters*, vol. 48, no. 21, 2021.
- [4] 赵玉娟 *et al.*, "基于 ConvLSTM 的改进雷达回波外推方法研究," *计算机测量与控制*, pp. 1-9.
- [5] 刘黎平, 葛润生, and 张沛源, "双线偏振多普勒天气雷达遥测降水强度和液态含水量的方法和精度研究," *大气科学*, no. 05, pp. 709-720, 2002.
- [6] 郑越, "基于 CNN 的民航天气雷达图像中降水及雷暴识别研究," 硕士, 云南大学, 2019.
- [7] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997.
- [8] S. Kim, S. Hong, M. Joh, and S. K. Song, "DeepRain: ConvLSTM Network for Precipitation Prediction using Multichannel Radar Data," 2017.
- [9] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon. (2018). *CBAM: Convolutional Block Attention Module*.

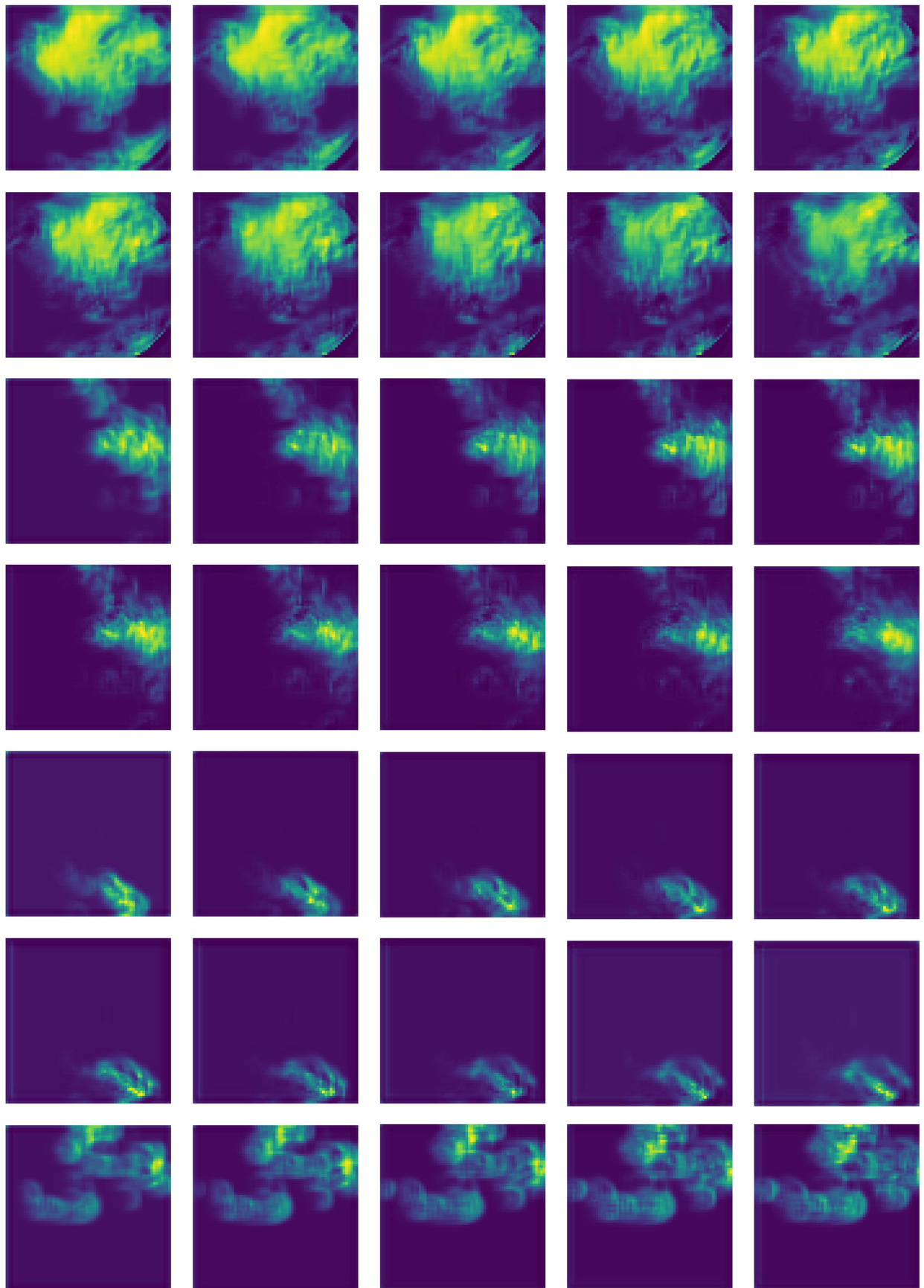
附录 A:部分预测结果图片

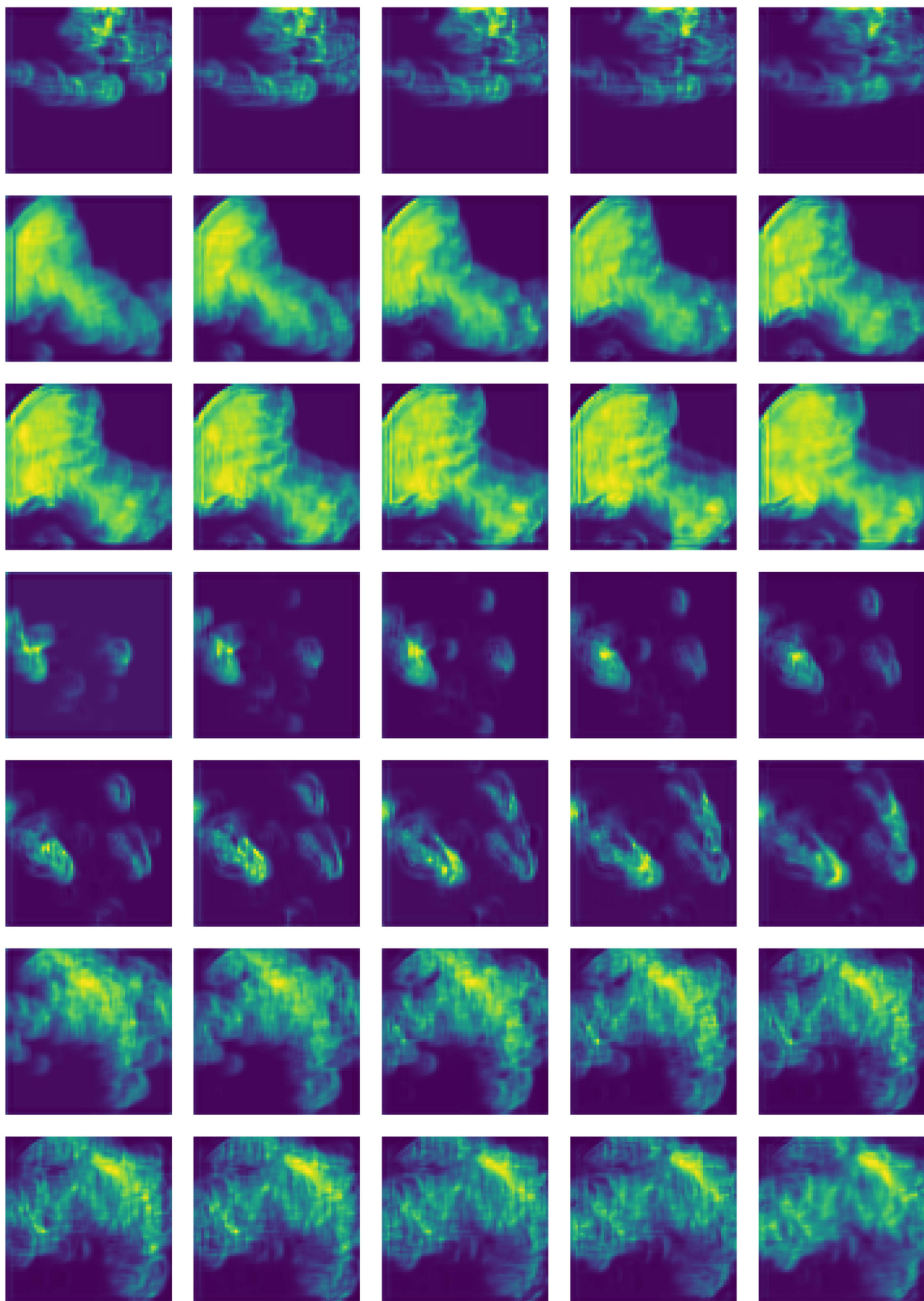


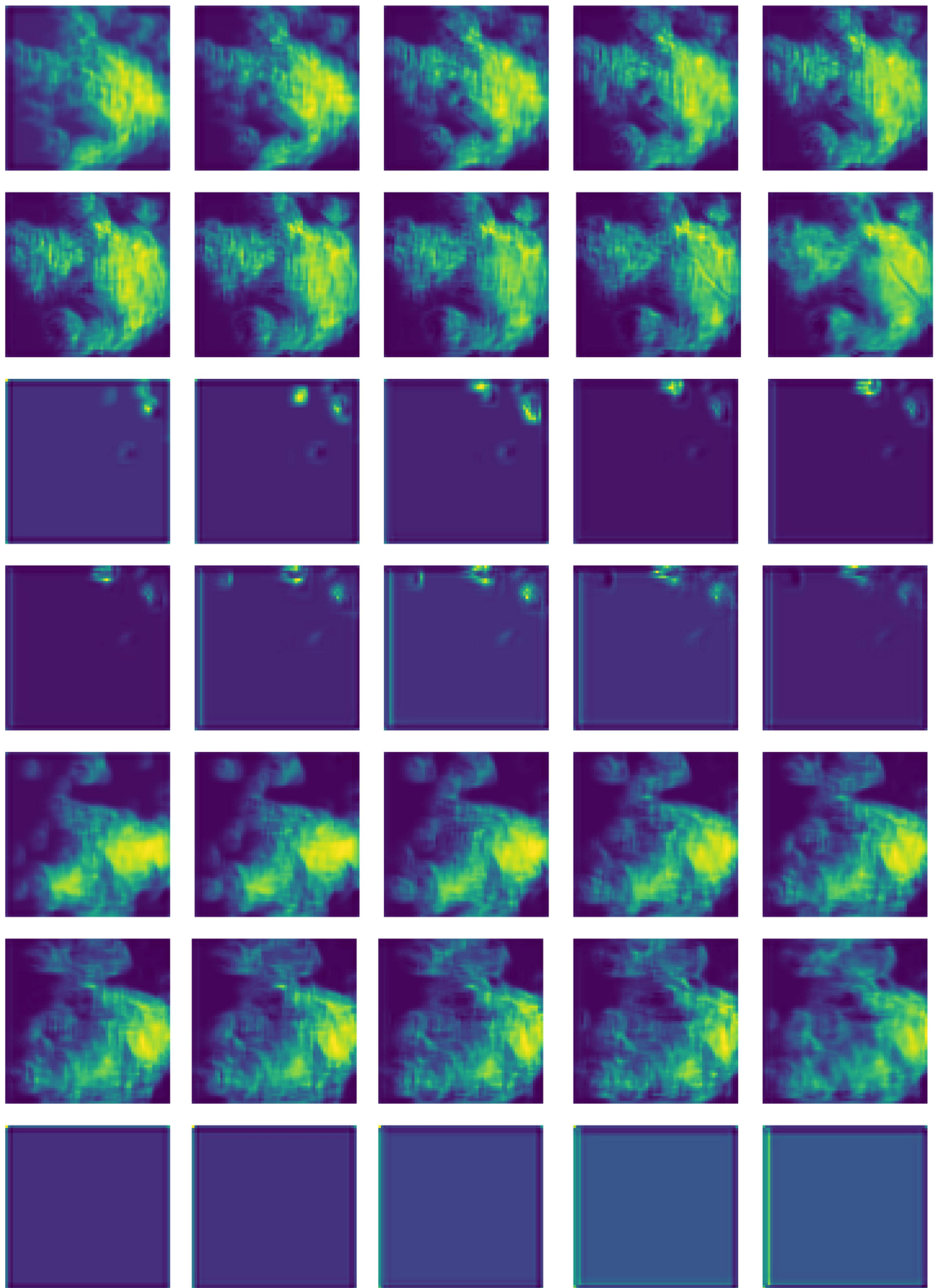


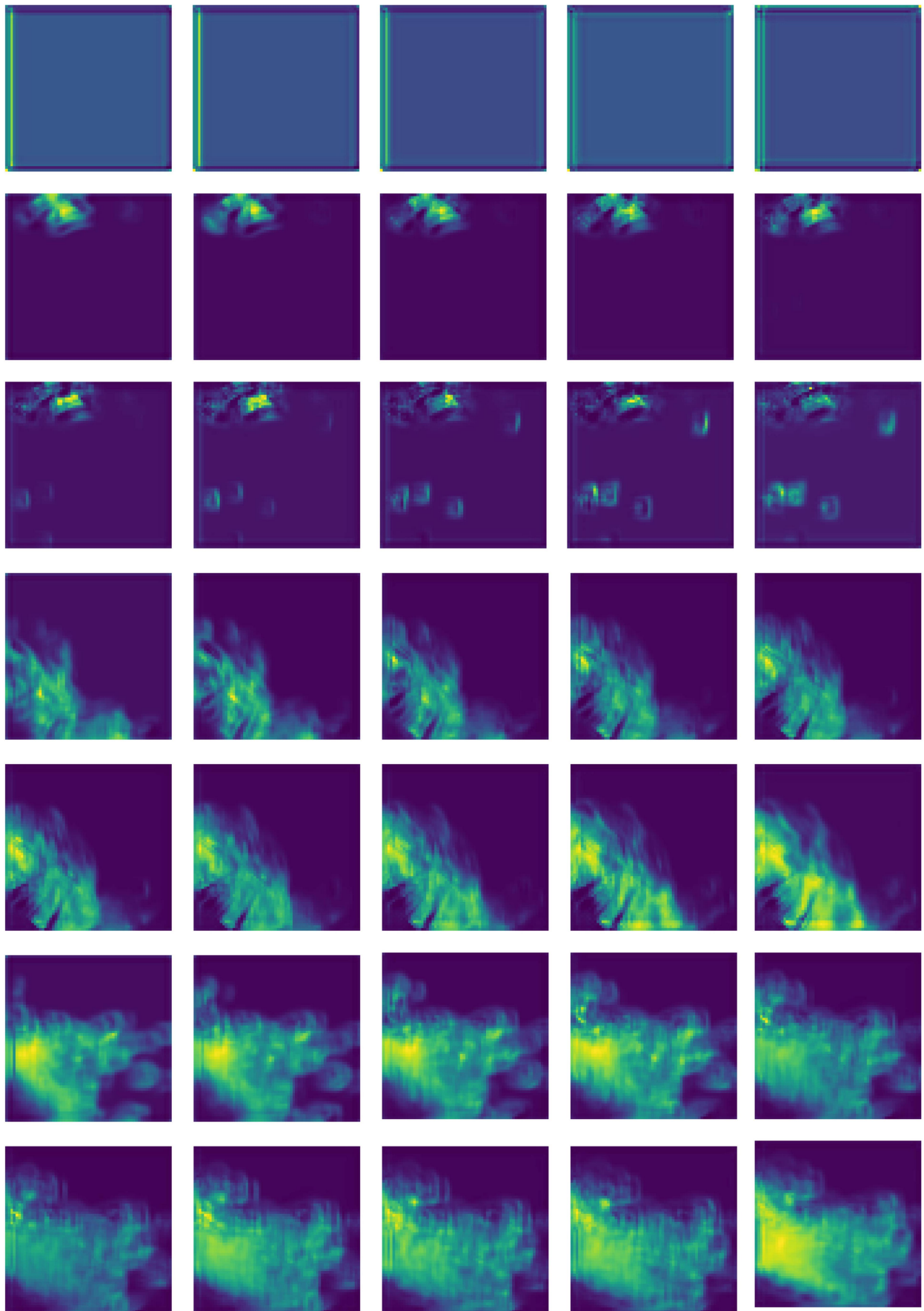


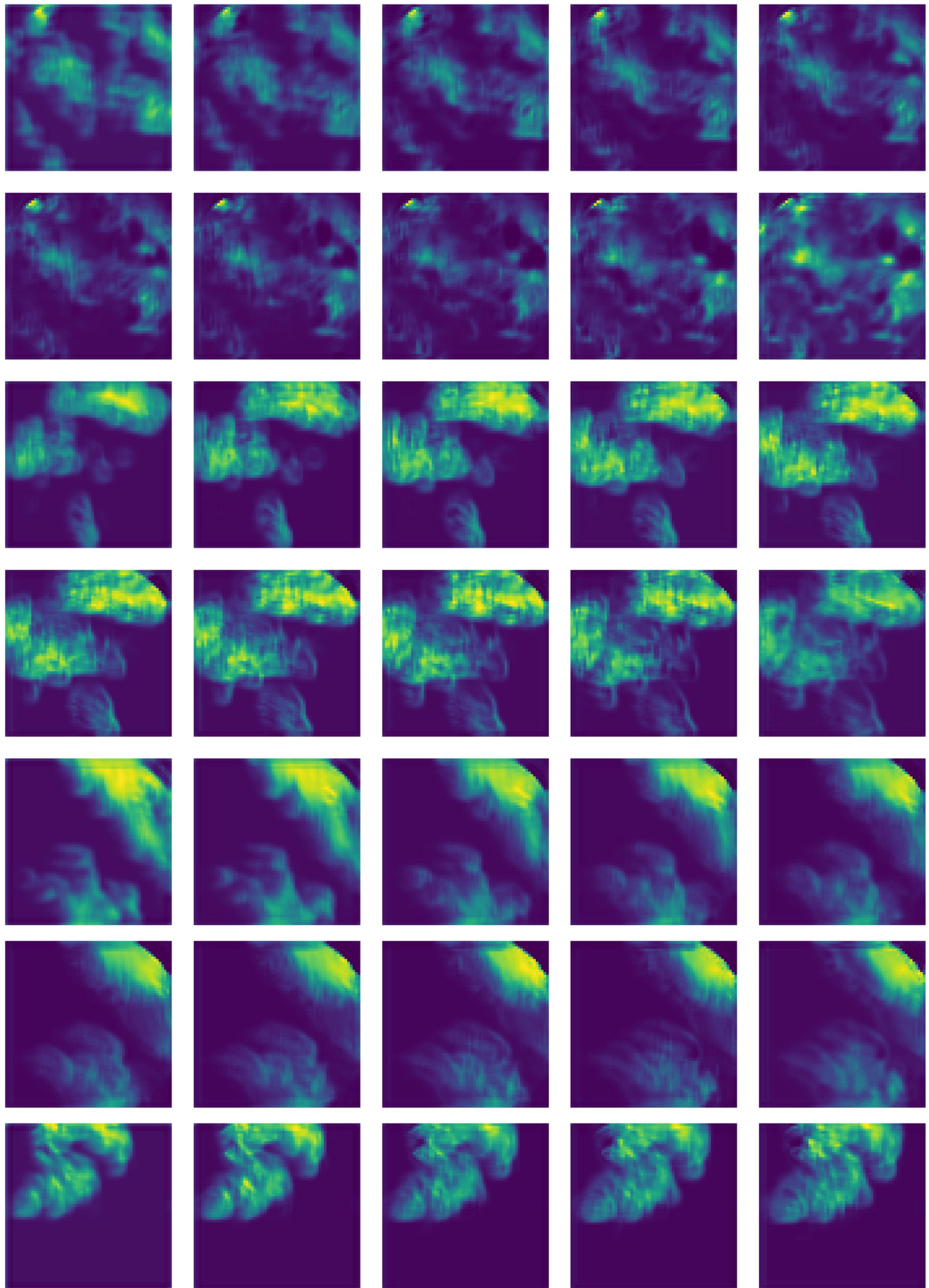


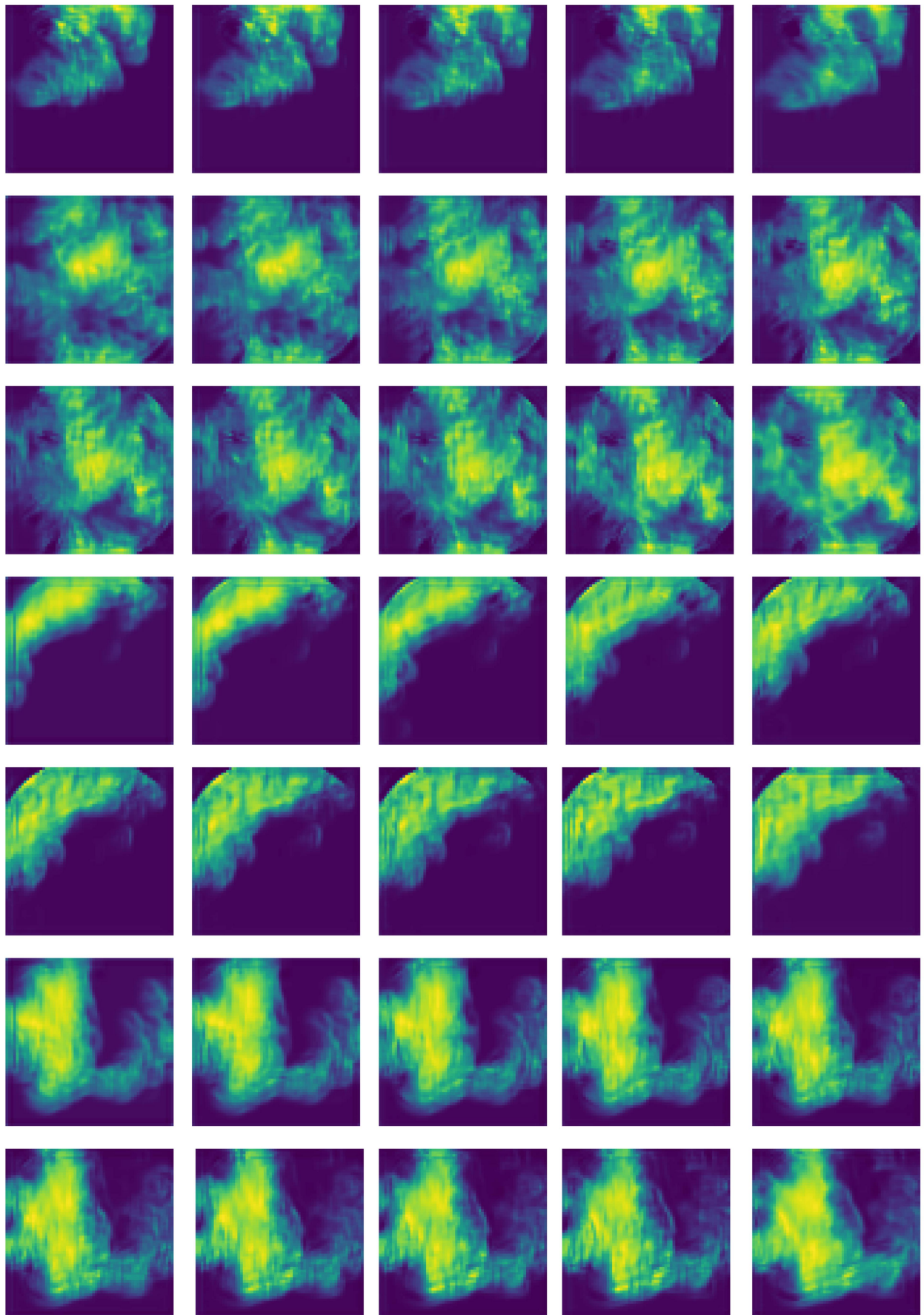












附录 B:代码

```
import os

import numpy as np

import matplotlib.pyplot as plt

import time

from tensorflow import keras

from tensorflow.keras.layers import Input, ConvLSTM2D,
concatenate, BatchNormalization, Conv3D

from tensorflow.keras.models import Model


def avgepool(image, pooling_factor):
    # 4 表示将尺寸缩小为原来的 1/4，从 256x256 变为 64x64
    # 计算新图像的大小
    new_height = image.shape[0] // pooling_factor
    new_width = image.shape[1] // pooling_factor
    # 使用循环来执行平均池化
    pooled_image = np.zeros((new_height, new_width), dtype=image.dtype)
    for i in range(new_height):
        for j in range(new_width):
            # 计算每个池化窗口的平均值
            window = image[i * pooling_factor:(i + 1) * pooling_factor, j * pooling_factor:(j
+ 1) * pooling_factor]
            pooled_value = np.mean(window)
            pooled_image[i, j] = pooled_value
    return pooled_image


def dataloder(folder_path, n, pool_n):
    # 指定根目录
```

```

# n:截取帧数
file_fi_list = [f for f in os.listdir(folder_path)]
allfile = []
for j, file_fi_dir in enumerate(file_fi_list[:100]):
    file_list = [j for j in os.listdir(os.path.join(folder_path,file_fi_dir))]
    data_list = []
    for i, file_name in enumerate(file_list[:n]):
        file_path = os.path.join(os.path.join(folder_path, file_fi_dir), file_name)
        data = np.load(file_path)
        data = avgpool(data,pool_n)
        data_list.append(data)
        # 在这里可以对数据进行处理或分析
        # 例如， 你可以打印数据的形状
        print(f'Loaded {file_name}')
    allfile.append(data_list)
    print(f'Loaded {file_fi_dir}')
allfile = np.array(allfile)
return (allfile)

folder_pathdbz = r'E:\NJUCPOL\NJU_CPOL_update2308\dBZ\3.0km' # 替换为实际文件夹的路径
folder_pathkdp = r'E:\NJUCPOL\NJU_CPOL_update2308\KDP\3.0km' # 替换为实际文件夹的路径
folder_pathzdr = r'E:\NJUCPOL\NJU_CPOL_update2308\ZDR\3.0km' # 替换为实际文件夹的路径

dBZ_data = dataloder(folder_pathdbz,20,4)
KDP_data = dataloder(folder_pathkdp,20,4)
ZDR_data = dataloder(folder_pathzdr,20,4)

```

```

def datashow(data):
    #此函数显示一个样本的帧数变化
    for i in range(0, data.shape[0]):
        plt.imshow(data[i], cmap='viridis') # cmap 参数用于指定颜色映射
        # 添加颜色条（可选）
        plt.colorbar()
        # 显示图像
        plt.show()
        time.sleep(0.3) # 延时 5 秒钟
        plt.close()

```

```

datashow(dBZ_data[0])

```

```

def channel_attention(input_feature, ratio=8):

    channel_axis = 1 if K.image_data_format() == "channels_first" else -1
    channel = input_feature._keras_shape[channel_axis]

    shared_layer_one = Dense(channel//ratio,
                              kernel_initializer='he_normal',
                              activation = 'relu',
                              use_bias=True,
                              bias_initializer='zeros')

    shared_layer_two = Dense(channel,
                              kernel_initializer='he_normal',
                              use_bias=True,
                              bias_initializer='zeros')

```

```

avg_pool = GlobalAveragePooling2D()(input_feature)
avg_pool = Reshape((1,1,channel))(avg_pool)
assert avg_pool._keras_shape[1:] == (1,1,channel)
avg_pool = shared_layer_one(avg_pool)
assert avg_pool._keras_shape[1:] == (1,1,channel//ratio)
avg_pool = shared_layer_two(avg_pool)
assert avg_pool._keras_shape[1:] == (1,1,channel)

max_pool = GlobalMaxPooling2D()(input_feature)
max_pool = Reshape((1,1,channel))(max_pool)
assert max_pool._keras_shape[1:] == (1,1,channel)
max_pool = shared_layer_one(max_pool)
assert max_pool._keras_shape[1:] == (1,1,channel//ratio)
max_pool = shared_layer_two(max_pool)
assert max_pool._keras_shape[1:] == (1,1,channel)

cbam_feature = Add()([avg_pool,max_pool])
cbam_feature = Activation('hard_sigmoid')(cbam_feature)

```

```

if K.image_data_format() == "channels_first":
    cbam_feature = Permute((3, 1, 2))(cbam_feature)

```

```

return multiply([input_feature, cbam_feature])

```

空间注意力机制

```

def spatial_attention(input_feature):
    kernel_size = 7
    if K.image_data_format() == "channels_first":
        channel = input_feature._keras_shape[1]

```

```

        cbam_feature = Permute((2,3,1))(input_feature)
    else:
        channel = input_feature._keras_shape[-1]
        cbam_feature = input_feature

    avg_pool = Lambda(lambda x: K.mean(x, axis=3, keepdims=True))(cbam_feature)
    assert avg_pool._keras_shape[-1] == 1
    max_pool = Lambda(lambda x: K.max(x, axis=3, keepdims=True))(cbam_feature)
    assert max_pool._keras_shape[-1] == 1
    concat = Concatenate(axis=3)([avg_pool, max_pool])
    assert concat._keras_shape[-1] == 2
    cbam_feature = Conv2D(filters = 1,
                          kernel_size=kernel_size,
                          activation = 'hard_sigmoid',
                          strides=1,
                          padding='same',
                          kernel_initializer='he_normal',
                          use_bias=False)(concat)
    assert cbam_feature._keras_shape[-1] == 1

    if K.image_data_format() == "channels_first":
        cbam_feature = Permute((3, 1, 2))(cbam_feature)

    return multiply([input_feature, cbam_feature])

def cbam_block(cbam_feature, ratio=8):
    """Contains the implementation of Convolutional Block Attention Module(CBAM) block.
    As described in CBAM: Convolutional Block Attention Module. """

```



```

cbam_feature = channel_attention(cbam_feature, ratio)
cbam_feature = spatial_attention(cbam_feature, )
return cbam_feature

```

```

def normal(data4d):
    num_samples, channels, height, width = data4d.shape
    normalized = np.zeros((num_samples, channels, height, width))
    # 遍历数组中的每个二维矩阵并进行归一化
    for i in range(num_samples):
        for j in range(channels):
            matrix_2d = data4d[i, j, :, :]
            # 进行归一化操作，例如最小-最大归一化
            min_value = np.min(matrix_2d)
            max_value = np.max(matrix_2d)
            normalized_matrix = np.divide((matrix_2d-min_value),(max_value-min_value))
            # 将归一化后的矩阵放回原位置
            normalized[i, j, :, :] = normalized_matrix
    normalized = np.nan_to_num(normalized)
    return normalized

```

```

dBZ_data_dataset = normal(dBZ_data)
KDP_data_dataset = normal(KDP_data)
ZDR_data_dataset = normal(ZDR_data)
datashow(ZDR_data_dataset[0])

```

```

dBZ_data_datasets = np.expand_dims(dBZ_data_dataset, axis=-1)
KDP_data_datasets = np.expand_dims(KDP_data_dataset, axis=-1)
ZDR_data_datasets = np.expand_dims(ZDR_data_dataset, axis=-1)

```

```

indexes = np.arange(dBZ_data_datasets.shape[0])
np.random.shuffle(indexes) # 打乱索引顺序
# 训练集：测试集
train_index = indexes[: int(0.666 * dBZ_data_datasets.shape[0])]
val_index = indexes[int(0.666 * dBZ_data_datasets.shape[0]):]

```

```

train_dataset_D = dBZ_data_datasets[train_index]
val_dataset_D = dBZ_data_datasets[val_index]
train_dataset_K = KDP_data_datasets[train_index]
val_dataset_K = KDP_data_datasets[val_index]
train_dataset_Z = ZDR_data_datasets[train_index]
val_dataset_Z = ZDR_data_datasets[val_index]

```

```

print(train_dataset_D.shape)
print(train_dataset_D.shape)

```

```

def create_shifted_frames(data):
    x = data[:, 0: 10, :, :]
    y = data[:, 10: 20, :, :]
    return x, y
x_train1, y_train1 = create_shifted_frames(train_dataset_D)
x_val1, y_val1 = create_shifted_frames(train_dataset_D)
x_train2, y_train2 = create_shifted_frames(train_dataset_K)
x_val2, y_val2 = create_shifted_frames(train_dataset_K)
x_train3, y_train3 = create_shifted_frames(train_dataset_Z)
x_val3, y_val3 = create_shifted_frames(train_dataset_Z)

```

定义三个输入通道的大小为 (timesteps, height, width, channels) 的数据

```
input1 = Input(shape=(10, 64, 64, 1))
```

```
input2 = Input(shape=(10, 64, 64, 1))
```

```
input3 = Input(shape=(10, 64, 64, 1))
```

第一个输入通道的 ConvLSTM 层

```
convlstm1 = ConvLSTM2D(filters=32, kernel_size=(3, 3), activation='relu',  
return_sequences=True, padding='same')(input1)
```

```
BN1 = BatchNormalization()(convlstm1)
```

```
convlstm2 = ConvLSTM2D(filters=32, kernel_size=(3, 3), activation='relu',  
return_sequences=True, padding='same')(BN1)
```

```
BN2 = BatchNormalization()(convlstm2)
```

```
convlstm3 = ConvLSTM2D(filters=64, kernel_size=(3, 3), activation='relu',  
return_sequences=True, padding='same')(BN2)
```

第二个输入通道的 ConvLSTM 层

```
convlstm1_2 = ConvLSTM2D(filters=32, kernel_size=(3, 3), activation='relu',  
return_sequences=True, padding='same')(input2)
```

```
BN1_2 = BatchNormalization()(convlstm1_2)
```

```
convlstm2_2 = ConvLSTM2D(filters=32, kernel_size=(3, 3), activation='relu',  
return_sequences=True, padding='same')(BN1_2)
```

```
BN2_2 = BatchNormalization()(convlstm2_2)
```

```
convlstm3_2 = ConvLSTM2D(filters=64, kernel_size=(3, 3), activation='relu',  
return_sequences=True, padding='same')(BN2_2)
```

第三个输入通道的 ConvLSTM 层

```
convlstm1_3 = ConvLSTM2D(filters=32, kernel_size=(3, 3), activation='relu',  
return_sequences=True, padding='same')(input3)
```

```

BN1_3 = BatchNormalization()(convlstm1_3)
convlstm2_3 = ConvLSTM2D(filters=32, kernel_size=(3, 3), activation='relu',
return_sequences=True, padding='same')(BN1_3)
BN2_3 = BatchNormalization()(convlstm2_3)
convlstm3_3 = ConvLSTM2D(filters=64, kernel_size=(3, 3), activation='relu',
return_sequences=True, padding='same')(BN2_3)
# 将三个 ConvLSTM 层的输出连接在一起
concatenated = concatenate([convlstm3, convlstm3_2, convlstm3_3], axis=-1)
# 添加其他层，例如卷积层、池化层、全连接层等
# 例如，添加一个池化层和全连接层
output = Conv3D(filters=1, kernel_size=(3, 3, 3),
activation='sigmoid',
padding='same', data_format='channels_last')(concatenated)
# 创建模型
model = Model(inputs=[input1, input2, input3], outputs=output)
# 编译模型
model.compile(optimizer='adam', loss='mean_squared_error')
# 打印模型结构
model.summary()

epochs = 50
batch_size = 4

early_stopping = keras.callbacks.EarlyStopping(monitor="val_loss", patience=10)
reduce_lr = keras.callbacks.ReduceLROnPlateau(monitor="val_loss", patience=5)

# 拟合模型.
history = model.fit(
    [x_train1, x_train2, x_train3],

```

```

        y_train1,
        batch_size=batch_size,
        epochs=epochs,
        validation_data=([x_val1, x_val2, x_val3], y_val1),
        callbacks=[early_stopping, reduce_lr],
    )
    model.summary()

```

```

x = model.predict((train_dataset_D[:,0:10]),(train_dataset_K[:,0:10]),(train_dataset_Z[:,0:10]))

```

```

datashow(x[0])

```

```

datashow(train_dataset_D[1])

```

```

x = np.zeros((50, 10, 64, 64, 1))

```

```

for i in range (50):

```

```

    x[i] =

```

```

    model.predict([np.expand_dims(train_dataset_D[i,0:10],axis=0),np.expand_dims(train_dataset_
K[i,0:10],axis = 0),np.expand_dims(train_dataset_Z[i,0:10],axis = 0)])

```

```

image_array = x

```

```

# Create a directory to save the images

```

```

save_directory = r'E:\NJUCPOL\result'

```

```

os.makedirs(save_directory, exist_ok=True)

```

```

# Iterate over the images

```

```

for i in range(image_array.shape[0]):

```

```

    for j in range(image_array.shape[1]):

```

```

# Get the current image

image = image_array[i, j, :, :, 0] # Assuming your images are grayscale

# Plot the image

plt.imshow(image)

plt.axis('off')

# Save the image

filename = f'image_{i}_{j}.png'

save_path = os.path.join(save_directory, filename)

plt.savefig(save_path, bbox_inches='tight')

# Clear the plot for the next image

plt.clf()

# Print a message when the process is complete

print('Images saved successfully!')


# for i in range(ZDR_data[0].shape[0]):
#     # 创建一个新的图像
#     plt.figure()
#     # 显示图像， cmap 根据您的数据类型选择合适的颜色映射
#     plt.imshow(ZDR_data[0][i]) # 这里使用灰度图
#     # 关闭坐标轴（可选）
#     plt.axis('off')
#     # 保存图像，可以根据需要修改文件名和文件格式
#     plt.savefig(f'image_ZDR{i + 1}.png', dpi=300, bbox_inches='tight')
#     # 关闭当前图像
#     plt.close()
#
#
# def image_entropy(image):
#     # 将图像矩阵拉平为一维数组

```

```

#     flat_image = image.ravel()
#     # 计算图像直方图
#     histogram, _ = np.histogram(flat_image, bins=256, range=(0, 1))
#     # 将直方图值归一化为概率分布
#     histogram = histogram / float(flat_image.size)
#     # 计算图像熵
#     entropy = -np.sum(histogram * np.log2(np.maximum(histogram, 1e-10)))
#     return entropy
#
# image_entropies2 = []
# for i in range(KDP_data[0][0:10].shape[0]):
#     entropy = image_entropy(KDP_data[0][i])
#     image_entropies2.append(entropy)
#
# correlation_coefficientbc = np.corrcoef(image_entropies2, image_entropies1)
#
# plt.figure(figsize=(5, 3))
# plt.imshow(correlation_coefficientab, 'Greys')
# plt.axis('off')
# # 保存图像，可以根据需要修改文件名和文件格式
# plt.savefig(f'correlation_coefficientab.png', dpi=300, bbox_inches='tight')
# plt.close()
#
# # import seaborn as sns
#
# plt.figure(figsize=(10, 8))
# sns.set(font_scale=2)
# sns.heatmap(correlation_coefficientbc, annot=True, cmap='coolwarm', fmt=".2f")
# plt.axis('off')

```



```
# plt.savefig(f'correlation_coefficientbc.png', dpi=300, bbox_inches='tight')
# plt.close()
```

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import font_manager as fm, rcParams
from sklearn.cross_decomposition import PLSRegression
from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error
import copy
```

```
# ref: https://blog.csdn.net/qq2512446791
```

```
def PC_Cross_Validation(X, y, pc, cv):
    """
        x :光谱矩阵 nxm
        y :浓度阵 （化学值）
        pc:最大主成分数
        cv:交叉验证数量

    return :
        RMSECV:各主成分数对应的 RMSECV
        PRESS :各主成分数对应的 PRESS
        rindex:最佳主成分数
    """
    kf = KFold(n_splits=cv)
    RMSECV = []
    for i in range(pc):
```

```

RMSE = []
for train_index, test_index in kf.split(X):
    x_train, x_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
    pls = PLSRegression(n_components=i + 1)
    pls.fit(x_train, y_train)
    y_predict = pls.predict(x_test)
    RMSE.append(np.sqrt(mean_squared_error(y_test, y_predict)))
RMSE_mean = np.mean(RMSE)
RMSECV.append(RMSE_mean)
rindex = np.argmin(RMSECV)
return RMSECV, rindex

```

```

def Cross_Validation(X, y, pc, cv):

```

```

    """
    x :雷达数据
    y :降雨矩阵
    pc:最大主成分数
    cv:交叉验证数量
    return :
        RMSECV:各主成分数对应的 RMSECV
    """
    kf = KFold(n_splits=cv)
    RMSE = []
    for train_index, test_index in kf.split(X):
        x_train, x_test = X[train_index], X[test_index]
        y_train, y_test = y[train_index], y[test_index]
        pls = PLSRegression(n_components=pc)
        pls.fit(x_train, y_train)

```

```

        y_predict = pls.predict(x_test)

        RMSE.append(np.sqrt(mean_squared_error(y_test, y_predict)))

    RMSE_mean = np.mean(RMSE)

    return RMSE_mean

def CARS_Cloud(X, y, N=50, f=20, cv=10):

    p = 0.8

    m, n = X.shape

    u = np.power((n/2), (1/(N-1)))

    k = (1/(N-1)) * np.log(n/2)

    cal_num = np.round(m * p)

    # val_num = m - cal_num

    b2 = np.arange(n)

    x = copy.deepcopy(X)

    D = np.vstack((np.array(b2).reshape(1, -1), X))

    WaveData = []

    # Coeff = []

    WaveNum = []

    RMSECV = []

    r = []

    for i in range(1, N+1):

        r.append(u*np.exp(-1*k*i))

        wave_num = int(np.round(r[i-1]*n))

        WaveNum = np.hstack((WaveNum, wave_num))

        cal_index = np.random.choice \

            (np.arange(m), size=int(cal_num), replace=False)

        wave_index = b2[:wave_num].reshape(1, -1)[0]

        xcal = x[np.ix_(list(cal_index), list(wave_index))]

        #xcal = xcal[:,wave_index].reshape(-1, wave_num)

```

```

ycal = y[cal_index]
x = x[:, wave_index]
D = D[:, wave_index]
d = D[0, :].reshape(1,-1)
wnum = n - wave_num
if wnum > 0:
    d = np.hstack((d, np.full((1, wnum), -1)))
if len(WaveData) == 0:
    WaveData = d
else:
    WaveData = np.vstack((WaveData, d.reshape(1, -1)))

if wave_num < f:
    f = wave_num

pls = PLSRegression(n_components=f)
pls.fit(xcal, ycal)
beta = pls.coef_
b = np.abs(beta)
b2 = np.argsort(-b, axis=0)
coef = copy.deepcopy(beta)
coeff = coef[b2, :].reshape(len(b2), -1)
rmsecv, rindex = PC_Cross_Validation(xcal, ycal, f, cv)
RMSECV.append(Cross_Validation(xcal, ycal, rindex+1, cv))

```

```

WAVE = []

```

```

for i in range(WaveData.shape[0]):
    wd = WaveData[i, :]

```

```

# cd = CoeffData[i, :]
WD = np.ones((len(wd)))
# CO = np.ones((len(wd)))
for j in range(len(wd)):
    ind = np.where(wd == j)
    if len(ind[0]) == 0:
        WD[j] = 0
        # CO[j] = 0
    else:
        WD[j] = wd[ind[0]]
        # CO[j] = cd[ind[0]]
if len(WAVE) == 0:
    WAVE = copy.deepcopy(WD)
else:
    WAVE = np.vstack((WAVE, WD.reshape(1, -1)))

```

```

MinIndex = np.argmin(RMSECV)
Optimal = WAVE[MinIndex, :]
boindex = np.where(Optimal != 0)
OptWave = boindex[0]

```

```

return OptWave

```

```

import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt

```

```

# 定义分段拟合函数，这里使用两个线性段
def piecewise_linear(y, x, a1, b1, a2, b2):
    mask = y < 30 # 以 2.0 为分界点，将数据分为两段
    y1 = a1 * x[mask] + b1
    y2 = a2 * x[~mask] + b2
    return np.concatenate((y1, y2))

# 拟合数据
initial_guess = [1.0, 0.0, -1.0, 3.0] # 初始猜测参数
params, covariance = curve_fit(piecewise_linear, x, y, p0=initial_guess)

# 提取拟合后的参数
a1, b1, a2, b2 = params

# 绘制原始数据和分段拟合结果
plt.scatter(x, y, label='Data')
plt.plot(x, piecewise_linear(x, a1, b1, a2, b2), 'r-', label='Piecewise Fit')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.show()

```